

REFRESHER READING

2020 CFA PROGRAM • LEVEL II • READING 7

Quantitative Methods

Machine Learning

by Kathleen DeRose, CFA, and Christophe Le Lannou

Kathleen DeRose, CFA, is at New York University, Stern School of Business (USA).

Christophe Le Lannou is at dataLearning (United Kingdom).

LEARNING OUTCOMES

Mastery	The candidate should be able to:
☐	a. distinguish between supervised machine learning, unsupervised machine learning, and deep learning;
☐	b. describe overfitting and identify methods of addressing it;
☐	c. describe supervised machine learning algorithms—including penalized regression, support vector machine, k-nearest neighbor, classification and regression tree, ensemble learning, and random forest—and determine the problems for which they are best suited;
☐	d. describe unsupervised machine learning algorithms—including principal components analysis, k-means clustering, and hierarchical clustering—and determine the problems for which they are best suited;
☐	e. describe neural networks, deep learning nets, and reinforcement learning.

INTRODUCTION

1

Investment firms are increasingly using technology at every step of the investment management value chain—from improving their understanding of clients, to uncovering new sources of alpha, to executing trades more efficiently. Machine learning techniques, a central part of that technology, are the subject of this reading. These techniques first appeared in finance in the 1990s and have since flourished with the explosion of data and cheap computing power.

This reading provides a high-level view of machine learning (ML). It covers a selection of key ML algorithms and their investment applications. Investment practitioners should be equipped with a basic understanding of the types of investment problems that machine learning can address, an idea of how the algorithms work,

and the vocabulary to interact with machine learning and data science experts. While investment practitioners need not master the details and mathematics of machine learning, as domain experts in investments they can play an important role by being able to source appropriate model inputs, interpret model outputs, and translate outputs into appropriate investment actions.

Section 2 gives an overview of machine learning in investment management. Section 3 defines machine learning and the types of problems that can be addressed by supervised and unsupervised learning. Section 4 describes evaluating machine learning algorithm performance. Key supervised machine learning algorithms are covered in Section 5, while Section 6 describes key unsupervised machine learning algorithms. Neural networks, deep learning nets, and reinforcement learning are covered in Section 7. The reading concludes with a summary.

2

MACHINE LEARNING AND INVESTMENT MANAGEMENT

The growing volume and exploding diversity of data, as well as the increasing economic value of insights extracted from these data, have inspired a resurgence in data science. This newly evolving field combines mathematics, computer science, and business analytics. It also strikes out in a new direction that relies on learning—from basic learning functions that map relationships between variables, to advanced neural networks that mimic physical processes that absorb, order, and adapt to information.

Machine learning has theoretical and practical implications for investment management. For example, machine learning could potentially reshape received wisdom about asset risk premiums and reconfigure investment management business processes. Large data sets and learning models are already affecting investment management practices—from client profiling, to asset allocation, to stock selection, to portfolio construction and risk management, to trading.

Machine learning applications are at each step of the asset and wealth management value chain. Chatbots answer basic retirement savings questions, learning from their interactions with investors. Machine learning methods can be used to generate alpha signals used in security selection by creating a non-linear forecast for a single time series, by deriving a forecast from a suite of predefined factors, or even by choosing input signals from existing or newly found data. For example, researchers using textual analysis have found that year-over-year changes in annual (10K) and quarterly (10Q) filings, particularly negative changes in the management discussion and risk sections, can strongly predict equity returns.

Machine learning methods can help calculate target portfolio weights that incorporate client restrictions and then dynamically weight them to maximize a Sharpe ratio or better estimate the variance–covariance matrix by using such techniques as principal components analysis, which reduces the number of variables needed to explain the variation in the data. Research suggests that machine learning solutions outperform mean–variance optimization in portfolio construction. Machine learning techniques are already creating better order flow management tools with non-linear trading algorithms that reduce the costs of implementing portfolio decisions. These developments have caused an evolution in automated tools, automated processes, and algorithmically automated businesses, like robo-advising.

WHAT IS MACHINE LEARNING?

3

We now discuss some fundamental concepts of machine learning, including a definition of machine learning and an overview of key types of machine learning, such as supervised and unsupervised ML.

3.1 Defining Machine Learning

Statistical approaches and machine learning techniques both analyze observations to reveal some underlying process; however, they diverge in their assumptions, terminology, and techniques. Statistical approaches rely on foundational assumptions and explicit models of structure, such as observed samples that are assumed to be drawn from a specified underlying probability distribution. These *a priori* restrictive assumptions can fail in reality.

In contrast, machine learning seeks to extract knowledge from large amounts of data with no such restrictions. The goal of machine learning algorithms is to automate decision-making processes by generalizing (i.e., “learning”) from known examples to determine an underlying structure in the data. The emphasis is on the ability of the algorithm to generate structure or predictions from data without any human help. An elementary way to think of ML algorithms is to “find the pattern, apply the pattern.”

Machine learning techniques are better able than statistical approaches to handle problems with many variables (high dimensionality) or with a high degree of non-linearity. ML algorithms are particularly good at detecting change, even in highly non-linear systems, because they can detect the preconditions of a model’s break or anticipate the probability of a regime switch.

Machine learning is broadly divided into three distinct classes of techniques: supervised learning, unsupervised learning, and deep learning.

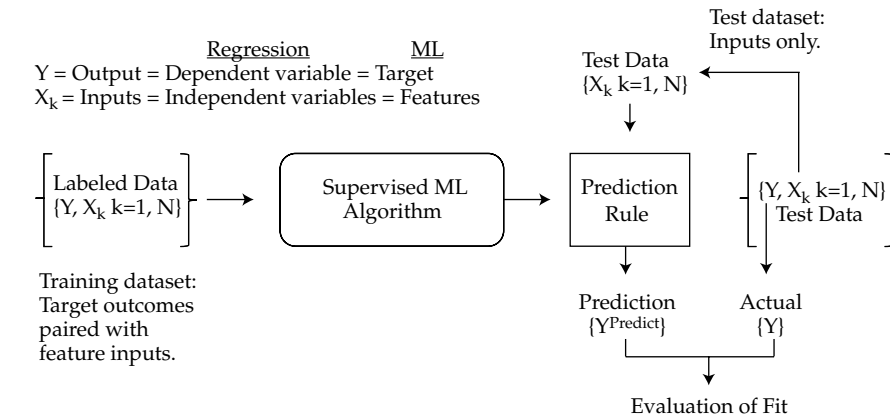
3.2 Supervised Learning

Supervised learning involves ML algorithms that infer patterns between a set of inputs (the X ’s) and the desired output (Y). The inferred pattern is then used to map a given input set into a predicted output. Supervised learning requires a **labeled data set**, one that contains matched sets of observed inputs and the associated output. Applying the ML algorithm to this data set to infer the pattern between the inputs and output is called “training” the algorithm. Once the algorithm has been trained, the inferred pattern can be used to predict output values based on new inputs (i.e., ones not in the training data set).

Multiple regression is an example of supervised learning. A regression model takes matched data (X ’s, Y) and uses it to estimate parameters that characterize the relationship between Y and the X ’s. The estimated parameters can then be used to predict Y on a new, different set of X ’s. The difference between the predicted and actual Y is used to evaluate how well the regression model predicts out-of-sample (i.e., using new data).

The terminology used with ML algorithms differs from that used in regression. Exhibit 1 provides a visual of the supervised learning model training process and a translation between regression and ML terminologies.

Exhibit 1 Overview of Supervised Learning



In supervised machine learning, the dependent variable (Y) is the **target** and the independent variables (X 's) are known as **features**. The labeled data (training data set) is used to train the supervised ML algorithm to infer a pattern-based prediction rule. The fit of the ML model is evaluated using labeled test data in which the predicted targets ($Y^{Predict}$) are compared to the actual targets (Y^{Actual}).

An example of supervised learning is the case in which ML algorithms are used to predict whether credit card transactions are fraudulent or legitimate. In the credit card example, the target is a binary variable with a value of 1 for “fraudulent” or 0 for “non-fraudulent.” The features are the transaction characteristics. The chosen ML algorithm uses these data to train a model to predict fraud more accurately in new transactions. The ML program “learns from experience” if the percentage of correctly predicted credit card transactions increases as the input from a growing credit card database increases. One possible ML algorithm to use here would be to fit a logistic regression model to the data to provide an estimate of the probability a transaction is fraudulent.

Supervised learning can be divided into two categories of problems, regression problems and classification problems, with the distinction between them being determined by the nature of the target (Y) variable. If the target variable is continuous, then the task is one of regression (even if the ML technique used is not “regression,” note this nuance of ML terminology). If the target variable is categorical or ordinal (i.e., a ranked category), then it is a classification problem. Regression and classification use different ML techniques.

Regression focuses on making predictions of continuous target variables. Most readers are already familiar with multiple linear regression (e.g., ordinary least squares) models, but other supervised learning techniques exist that include non-linear models. These non-linear models are useful for problems involving large data sets with large numbers of features, many of which may be correlated. Some examples of problems belonging to the regression category are using historical stock market returns to forecast stock price performance or using historical corporate financial ratios to forecast the probability of bond default.

Classification focuses on sorting observations into distinct categories. In a regression problem, when the dependent variable (target) is categorical, the model relating the outcome to the independent variables (features) is called a “classifier.” Many classification models are binary classifiers, as in the case of fraud detection for credit card transactions. Multi-category classification is not uncommon, as in the case of classifying firms into multiple credit rating categories. In assigning ratings, the outcome

variable is ordinal, meaning the categories have a distinct order or ranking (e.g., from low to high creditworthiness). Ordinal variables are intermediate between categorical variables and continuous variables on a scale of measurement.

3.3 Unsupervised Learning

Unsupervised learning is machine learning that does not make use of labeled data. More formally, in unsupervised learning, we have inputs (X 's) that are used for analysis without any target (Y) being supplied. In unsupervised learning, because the ML algorithm is not given labeled training data, the algorithm seeks to discover structure within the data themselves. As such, unsupervised learning is useful for exploring new data sets as it can provide human experts with insights into a data set too big or too complex to visualize.

Two important types of problems that are well suited to unsupervised machine learning are reducing the dimension of data and sorting data into clusters, known as dimension reduction and clustering, respectively.

Dimension reduction focuses on reducing the number of features while retaining variation across observations to preserve the information contained in that variation. Dimension reduction may have several purposes. It may be applied to data with a large number of features to produce a lower dimensional representation (i.e., with fewer features) that can fit, for example, on a computer screen. Dimension reduction is also used in many quantitative investment and risk management applications where it is critical to identify the major factors underlying asset price movements.

Clustering focuses on sorting observations into groups (clusters) such that observations in the same cluster are more similar to each other than they are to observations in other clusters. Groups are formed based on a set of criteria that may or may not be prespecified (such as the number of groups). Clustering has been used by asset managers to sort companies into empirically determined groupings (e.g., based on their financial statement data) rather than conventional groupings (e.g., based on sectors or countries).

3.4 Deep Learning and Reinforcement Learning

More broadly in the field of artificial intelligence, additional categories of machine learning algorithms are distinguished. In **deep learning**, sophisticated algorithms address highly complex tasks, such as image classification, face recognition, speech recognition, and natural language processing. In **reinforcement learning**, a computer learns from interacting with itself (or data generated by the same algorithm). Deep learning and reinforcement learning are themselves based on **neural networks** (NNs, also called *artificial neural networks*, or ANNs). NNs include highly flexible ML algorithms that have been successfully applied to a variety of tasks characterized by non-linearities and interactions among features. Besides being commonly used for classification and regression, neural networks are also the foundation for deep learning and reinforcement learning, which can be either supervised or unsupervised.

3.5 Summary of ML Algorithms and How to Choose Among Them

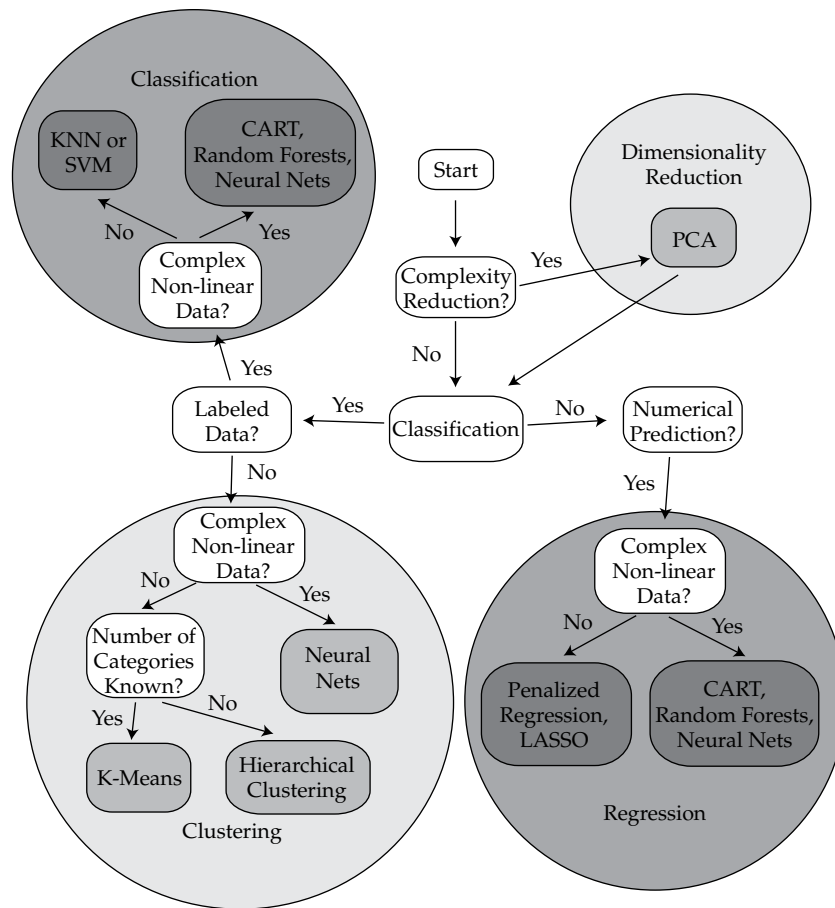
Exhibit 2 is a guide to the various machine learning algorithms organized by algorithm type (supervised or unsupervised) and by type of variables (continuous, categorical, or both). We will not cover linear or logistic regression since they are covered elsewhere

in quantitative methods. The extensions of linear regression, such as penalized regression and least absolute shrinkage and selection operator (LASSO), as well as the other ML algorithms shown in Exhibit 2, will be covered in this reading.

Exhibit 2 Guide to ML Algorithms

Variables	ML Algorithm Type	
	Supervised (Target Variable)	Unsupervised (No Target Variable)
Continuous	Regression • Linear; Penalized Regression/LASSO • Logistic • Classification and Regression Tree (CART) • Random Forest	Dimensionality Reduction • Principal Components Analysis (PCA)
		Clustering • K-Means • Hierarchical
Categorical	Classification • Logit • Support Vector Machine (SVM) • K-Nearest Neighbor (KNN) • Classification and Regression Tree (CART)	Dimensionality Reduction • Principal Components Analysis (PCA)
		Clustering • K-Means • Hierarchical
Continuous or Categorical	Neural Networks	Neural Networks
	Deep Learning	Deep Learning
	Reinforcement Learning	Reinforcement Learning

Exhibit 3 presents a stylized decision flowchart for choosing among the machine learning algorithms shown in Exhibit 2. The dark-shaded ovals contain the supervised ML algorithms; the light-shaded ovals contain the unsupervised ML algorithms; and the key questions to consider are shown in the unshaded boxes.

Exhibit 3 Stylized Decision Flowchart for Choosing ML Algorithms

First, start by asking, are the data complex, having many features that are highly correlated? If yes, then dimensionality reduction using principal components analysis (PCA) is appropriate.

Next, is the problem one of classification or numerical prediction? If numerical prediction, then depending on whether or not the data have non-linear characteristics, the choice of ML algorithms is from a set of regression algorithms—either penalized regression/LASSO for linear data or CART, random forest, and neural networks for non-linear data.

If the problem is one of classification, then depending on whether or not the data are labeled, the choice is either from a set of classification algorithms using labeled data or from a set of clustering algorithms using unlabeled data.

If the data are labeled, then depending on whether or not the data have non-linear characteristics, the choice of classification algorithm would be k-nearest neighbor (KNN) and support vector machine (SVM) for linear data or CART, random forest, and neural networks for non-linear data.

Finally, if the data are unlabeled, the choice of clustering algorithm depends on whether or not the data have non-linear characteristics. The choice of clustering algorithm would be neural networks for non-linear data or for linear data, k-means with a known number of categories, and hierarchical clustering with an unknown number of categories.

Following a description of how to evaluate ML algorithm performance, we will define all of the ML algorithms shown in Exhibit 3 and then explain their applications in investment management.

EXAMPLE 1**Machine Learning Overview**

- 1 Which of the following *best* describes machine learning? Machine learning:
 - A is a type of computer algorithm.
 - B is a set of computer-driven approaches aimed at generating structure or predictions from data without human intervention by finding a pattern and then applying the pattern.
 - C is a set of computer-driven approaches adapted to extracting information from linear, labeled data sets.
- 2 Which of the following statements is *most* accurate? When attempting to discover groupings of data without any target (Y) variable:
 - A an unsupervised ML algorithm is used.
 - B an ML algorithm that is given labeled training data is used.
 - C a supervised ML algorithm is used.
- 3 Which of the following statements concerning supervised learning *best* distinguishes it from unsupervised learning? Supervised learning involves:
 - A training on labeled data to infer a pattern-based prediction rule.
 - B training on unlabeled data to infer a pattern-based prediction rule.
 - C learning from unlabeled data by discovering underlying structure in the data themselves.
- 4 Which of the following *best* describes dimension reduction? Dimension reduction:
 - A focuses on classifying observations in a data set into known groups using labeled training data.
 - B focuses on clustering observations in a data set into unknown groups using unlabeled data.
 - C focuses on reducing the number of features in a data set while retaining variation across observations to preserve the information in that variation.

Solution to 1:

B is correct. A is incorrect because machine learning is not best described as a type of computer algorithm. C is incorrect because machine learning is not limited to extracting information from linear, labeled data sets.

Solution to 2:

A is correct. B is incorrect because the term “labeled training data” means the target (Y) is provided. C is incorrect because a supervised ML algorithm is meant to predict a target (Y) variable.

Solution to 3:

A is correct. B is incorrect because supervised learning uses labeled training data. C is incorrect because it describes unsupervised learning.

Solution to 4:

C is correct. A is incorrect because it describes classification, not dimension reduction. B is incorrect because it describes clustering, not dimension reduction.

OVERVIEW OF EVALUATING ML ALGORITHM PERFORMANCE

4

Machine learning algorithms promise several advantages relative to a structured statistical approach in exploring and analyzing the structure of very large data sets. ML algorithms have the ability to uncover complex interactions between feature variables and the target variable, and they can process massive amounts of data quickly. Moreover, many ML algorithms can easily capture non-linear relationships and may be able to recognize and predict structural changes between features and the target. These advantages mainly derive from the non-parametric and non-linear models that allow more flexibility when inferring relationships.

The flexibility of ML algorithms comes with a price, however. ML algorithms can produce overly complex models with results that are difficult to interpret, may be sensitive to particular changes in the data, and may fit the training data too well. An ML algorithm that fits the training data too well will typically not predict well using new data. This problem is known as **overfitting**, and it means that the fitted algorithm does not **generalize** well to new data. A model that generalizes well is a model that retains its explanatory power when predicting out-of-sample (i.e., using new data). An overfit model has incorporated the noise or random fluctuations in the training data into its learned relationship. The problem is that these aspects do not apply to any new data the algorithm receives and so will negatively impact the model's ability to generalize. The evaluation of any ML algorithm thus focuses on its prediction error on new data rather than on its goodness-of-fit on the data in which the algorithm was fitted (i.e., trained).

Generalization is an objective in model building, so the problem of overfitting is a challenge to attaining that objective. These two concepts are the focus of the discussion below.

4.1 Generalization and Overfitting

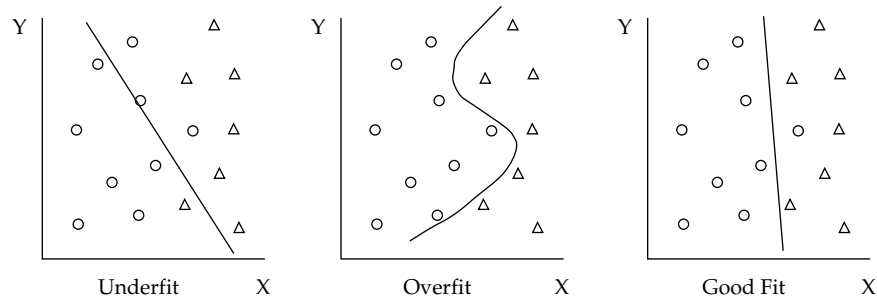
To properly describe generalization and overfitting of an ML model, it is important to note the partitioning of the data set to which the model will be applied. The data set is typically divided into three non-overlapping samples: 1) **training sample** used to train the model; 2) **validation sample** for validating and tuning the model; and 3) **test sample** for testing the model's ability to predict well on new data. The training sample is often referred to as being "in-sample," while the validation and test samples are commonly referred to as being "out-of-sample." We will return shortly to the topic of partitioning the data set.

To be valid and useful, any supervised machine learning model must generalize well beyond the training data. The model should retain its explanatory power when tested out-of-sample. As mentioned, one common reason for failure to generalize is overfitting. Think of overfitting as tailoring a custom suit that fits only one person. Continuing the analogy, underfitting is similar to making a baggy suit that fits no one, whereas robust fitting, the desired result, is similar to fashioning a universal suit that fits all similar people.

The concepts of underfitting, overfitting, and good (or robust) fitting are illustrated in Exhibit 4. Underfitting means the model does not capture the relationships in the data. The left graph shows four errors in this underfit model (three misclassified circles and one misclassified triangle). Overfitting means training a model to such a degree of specificity to the training data that the model begins to incorporate noise coming from quirks or spurious correlations; it mistakes randomness for patterns and relationships. The algorithm may have memorized the data, rather than learned from it, so it has

perfect hindsight but no foresight. The main contributors to overfitting are thus high noise levels in the data and too much complexity in the model. The middle graph shows no errors in this overfit model. **Complexity** refers to the number of features, terms, or branches in the model and to whether the model is linear or non-linear (non-linear is more complex). As models become more complex, overfitting risk increases. A good fit/robust model fits the training (in-sample) data well and generalizes well to out-of-sample data, both within acceptable degrees of error. The right graph shows that the good fitting model has only one error, the misclassified circle.

Exhibit 4 Under-, Over-, and Good-Fitting



4.2 Errors and Overfitting

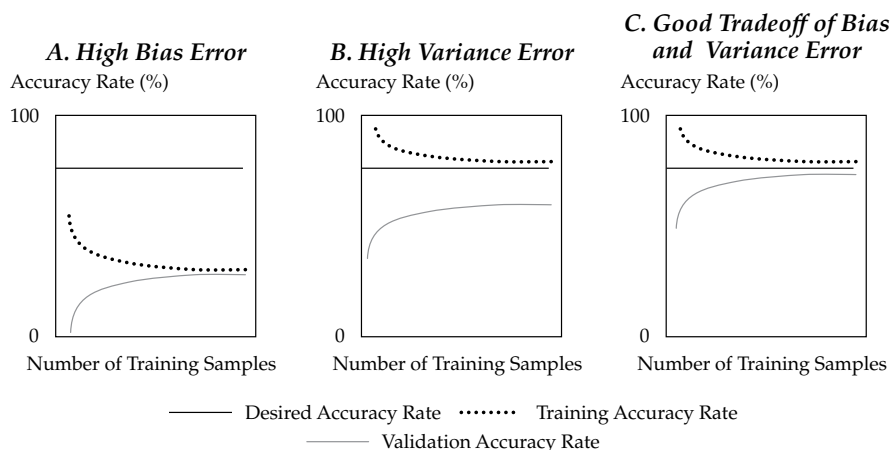
To capture these effects and calibrate degree of fit, data scientists compare error rates in- and out-of-sample as a function of both the data and the algorithm. Total in-sample errors (E_{in}) are generated by the predictions of the fitted relationship relative to actual target outcomes on the training sample. Total out-of-sample errors (E_{out}) are from either the validation or test samples. Low or no in-sample error but large out-of-sample error are indicative of poor generalization. Data scientists decompose the total out-of-sample error into three sources:

- 1 **Bias error**, or the degree to which a model fits the training data. Algorithms with erroneous assumptions produce high bias with poor approximation, causing underfitting and high in-sample error.
- 2 **Variance error**, or how much the model's results change in response to new data from validation and test samples. Unstable models pick up noise and produce high variance, causing overfitting and high out-of-sample error.
- 3 **Base error** due to randomness in the data.

A **learning curve** plots the accuracy rate ($= 1 - \text{error rate}$) in the validation or test samples (i.e., out-of-sample) against the amount of data in the training sample, so it is useful for describing under- and overfitting as a function of bias and variance errors. If the model is robust, out-of-sample accuracy increases as the training sample size increases. This implies that error rates experienced in the validation or test samples (E_{out}) and in the training (E_{in}) sample converge toward each other and toward a desired error rate (or, alternatively, the base error). In an underfitted model with high bias error, shown in the left panel of Exhibit 5, high error rates cause convergence below the desired accuracy rate. Adding more training samples will not improve the model. In an overfitted model with high variance error, shown in the middle panel of Exhibit 5, the validation sample and training sample error rates fail to converge. In

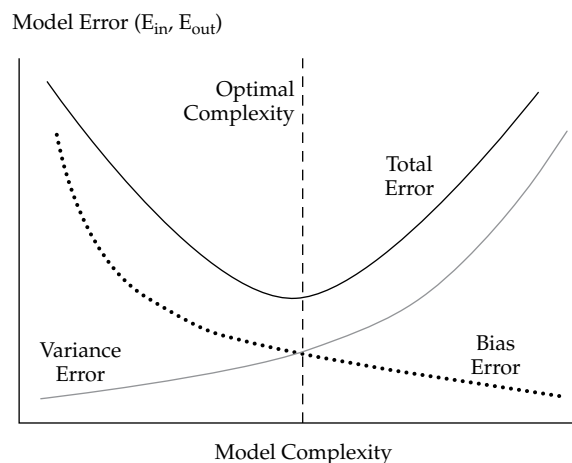
building models, data scientists try to simultaneously minimize both bias and variance errors while selecting an algorithm with good predictive or classifying power, as seen in the right panel of Exhibit 5.

Exhibit 5 Learning Curves: Accuracy in Validation and Training Samples vs. Training Sample Size



Out-of-sample error rates are also a function of model complexity. As complexity increases in the training set, error rates (E_{in}) fall and bias error shrinks. As complexity increases in the test set, however, error rates (E_{out}) rise and variance error rises. Typically, linear functions are more susceptible to bias error and underfitting, while non-linear functions are more prone to variance error and overfitting. Therefore, an optimal point of model complexity exists where the bias and variance error curves intersect and in- and out-of-sample error rates are minimized. A **fitting curve**, which shows in- and out-of-sample error rates (E_{in} and E_{out}) on the y-axis plotted against model complexity on the x-axis, is presented in Exhibit 6 and illustrates this trade-off.

Exhibit 6 Fitting Curve Shows Trade-Off Between Bias and Variance Errors and Model Complexity



Finding the optimal point (managing overfitting risk)—the sweet spot just before the total error rate starts to rise (due to increasing variance error)—is a core part of the machine learning process and the key to successful generalization. Data scientists express the trade-off between overfitting and generalization as a trade-off between *cost* (the difference between in- and out-of-sample error rates) and *complexity*. They use the trade-off between cost and complexity to calibrate and visualize under- and overfitting and to optimize their models.

4.3 Preventing Overfitting in Supervised Machine Learning

We have seen that overfitting impairs generalization, but overfitting potential is endemic to the supervised machine learning process due to the presence of noise. So, how do data scientists combat this risk? Two common guiding principles and two methods are used to reduce overfitting: 1) preventing the algorithm from getting too complex during selection and training, which requires estimating an overfitting penalty; and 2) proper data sampling achieved by using **cross-validation**, a technique for estimating out-of-sample error directly by determining the error in validation samples.

The first strategy comes from “Occam’s razor,” the problem-solving principle that the simplest solution tends to be the correct one. In supervised machine learning, it means limiting the number of features and penalizing algorithms that are too complex or too flexible by constraining them to include only parameters that reduce out-of-sample error.

The second strategy comes from the principle of avoiding sampling bias. Good experimental designs have representative samples and control groups, and both should be drawn from the same theoretical distribution. But sampling bias can creep into machine learning in many ways. The challenge is having a large enough data set to make both training and testing possible on representative samples. An unrepresentative sample or reducing the training sample size too much could obscure its true patterns, thereby increasing bias. In supervised machine learning, the technique for reducing sampling bias is through careful partitioning of the data set into three groups: 1) training sample, the set of labeled training data where the target variable (Y) is known; 2) validation sample, the set of data for comparing various solutions and tuning the selected model, thereby validating the model; and 3) test sample, the set of data held aside for testing to confirm the model’s predictive or classifying power. The goal, of course, is to deploy the tested model on fresh data from the same domain.

To mitigate the problem of such **holdout samples** (i.e., data samples not used to train the model) reducing the training set size too much, modelers use special cross-validation techniques. One such technique is **k-fold cross-validation**, in which the data (excluding test sample and fresh data) are shuffled randomly and then are divided into k equal sub-samples, with $k - 1$ samples used as training samples and one sample, the k th, used as a validation sample. Note that k is typically set at 5 or 10. This process is then repeated k times, which helps minimize both bias and variance by insuring that each data point is used in the training set $k - 1$ times and in the validation set once. The average of the k validation errors (mean E_{val}) is then taken as a reasonable estimate of the model’s out-of-sample error (E_{out}).

In sum, mitigating overfitting risk by avoiding excessive out-of-sample error is critical to creating a supervised machine learning model that generalizes well to fresh data sets drawn from the same domain. The main techniques used to mitigate overfitting risk in model construction are complexity reduction and cross-validation.

EXAMPLE 2**Evaluating ML Algorithm Performance**

Shreya Anand is a portfolio manager based in the Mumbai headquarters office of an investment firm, where she runs a high-dividend yield fund for wealthy clients. Anand has some knowledge of data science from her course in university. She is interested in classifying companies in the NIFTY 200 Index—an index of large- and mid-cap companies listed on the National Stock Exchange of India—into two categories: dividend increase and no dividend increase. She assembles data for training, validating, and testing an ML-based model that consists of 1,000 observations of NIFTY 200 companies, each consisting of 25 features (fundamental and technical) and the labeled target (dividend increase or no dividend increase).

After training her model, Anand discovers that while it is good at correctly classifying using the training sample, it does not perform well using new data. In consulting her colleagues about this issue, Anand hears conflicting explanations about what constitutes good generalization in an ML model:

- Statement 1 The model retains its explanatory power when predicting using new data (i.e., out-of-sample).
- Statement 2 The model shows low explanatory power after training using in-sample data (i.e., training data).
- Statement 3 The model loses its explanatory power when predicting using new data (i.e., out-of-sample).

- 1 Which statement made to Anand is most accurate?
 - A Statement 1.
 - B Statement 2.
 - C Statement 3.
- 2 Anand's model is *most likely* being impaired by which of the following:
 - A Underfitting and bias error
 - B Overfitting and variance error.
 - C Overfitting and bias error.
- 3 By implementing which one of the following actions can Anand address the problem?
 - A Estimate and incorporate into the model a penalty that decreases in size with the number of included features.
 - B Use the k-fold cross-validation technique to estimate the model's out-of-sample error, and then adjust the model accordingly.
 - C Use an unsupervised learning model.

Solution to 1:

A, Statement 1, is correct. B, Statement 2, is incorrect because it describes a poorly fitting model with high bias. C, Statement 3, is incorrect because it describes an overfitted model with poor generalization.

Solution to 2:

B is correct. Anand's model is good at correctly classifying using the training sample, but it does not perform well using new data. The model is overfitted, so it has high variance error.

Solution to 3:

B is correct. A is incorrect because the penalty should increase in size with the number of included features. C is incorrect because Anand is using labeled data for classification, and unsupervised learning models do not use labeled data.

5

SUPERVISED MACHINE LEARNING ALGORITHMS

Supervised machine learning models are trained using labeled data, and depending on the nature of the target (Y) variable, they can be divided into two types: regression for a continuous target variable and classification for a categorical or ordinal target variable. As shown in Exhibits 2 and 3 under regression, we will now cover penalized regression and LASSO. Then as shown under classification, we will introduce support vector machine (SVM), k-nearest neighbor (KNN), and classification and regression tree (CART) algorithms. Note that CART, as its name implies, can be used for both classification and regression problems.

In the following discussion, assume we have a number of observations of a target variable, Y , and n real-valued features, $X_1, \dots, X_n$, that we may use to establish a relationship (regression or classification) between X (a vector of the X_i) and Y for each observation in our data set.

5.1 Penalized Regression

Penalized regression is a computationally efficient technique used in prediction problems. In practice, penalized regression has been useful for reducing a large number of features to a manageable set and for making good predictions in a variety of large data sets, especially where features are correlated (i.e., when classical linear regression breaks down).

In a large data set context, we may have many features that potentially could be used to explain Y . When a model is fit to training data, the model may so closely reflect the characteristics of the specific training data that the model does not perform well on new data. Features may be included that reflect what is noise or randomness in the training data set that will not be present in new or future data used for making predictions. That is the problem of overfitting, and penalized regression can be described as a technique to avoid overfitting. In prediction, out-of-sample performance is key, so relatively parsimonious models (that is, models in which each variable plays an essential role) tend to work well because they are less subject to overfitting.

Let us suppose that we standardize our data, so the features have a mean of 0 and a variance of 1. Standardization of features will allow us to compare the magnitudes of regression coefficients for the feature variables. In ordinary linear regression (i.e., ordinary least squares, or OLS), the regression coefficients $\hat{b}_0, \hat{b}_1, \dots, \hat{b}_K$ are chosen to *minimize* the sum of the squared residuals (i.e., the sum of the squared difference between the actual values, Y_i , and the predicted values, $\hat{Y}_i$), or:

$$\sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Penalized regression includes a constraint such that the regression coefficients are chosen to minimize the sum of squared residuals *plus* a penalty term that increases in size with the number of included features. So, in a penalized regression, a feature

must make a sufficient contribution to model fit to offset the penalty from including it. Therefore, only the more important features for explaining Y will remain in the penalized regression model.

In one popular type of penalized regression, **LASSO** (least absolute shrinkage and selection operator), the penalty term has the following form, with $\lambda > 0$:

$$\text{Penalty term} = \lambda \sum_{k=1}^K |\hat{b}_k|.$$

In addition to minimizing the sum of the squared residuals, LASSO also involves minimizing the sum of the absolute values of the regression coefficients (see the following expression). The greater the number of included features (i.e., variables with non-zero coefficients), the larger the penalty term. Therefore, penalized regression ensures that a feature is included only if the sum of squared residuals declines by more than the penalty term increases. All types of penalized regression involve a trade-off of this type. Also, since LASSO eliminates the least important features from the model, it automatically performs feature selection.

$$\sum_{i=1}^n (Y_i - \hat{Y}_i)^2 + \lambda \sum_{k=1}^K |\hat{b}_k|$$

Lambda (λ) is a **hyperparameter**—a parameter whose value must be set by the researcher before learning begins—of the regression model and will determine the balance between fitting the model versus keeping the model parsimonious. Note that in the case where $\lambda = 0$, then the LASSO penalized regression is equivalent to an OLS regression. When using LASSO or other penalized regression techniques, the penalty term is added only during the model building process (i.e., when fitting the model to the training data). Once the model has been built, the penalty term is no longer needed, and the model is then evaluated by the sum of the squared residuals generated using the test data set.

With today's availability of fast computation algorithms, investment analysts are increasingly using LASSO and other regularization techniques to remove less pertinent features and build parsimonious models. **Regularization** describes methods that reduce statistical variability in high dimensional data estimation problems—in this case, reducing regression coefficient estimates toward zero and thereby avoiding complex models and the risk of overfitting. LASSO has been used, for example, for forecasting default probabilities in industrial sectors where scores of potential features, many collinear, have been reduced to fewer than 10 variables, which is important given the relatively small number (about 100) of observations of default.

Regularization methods can also be applied to non-linear models. A long-term challenge of the asset management industry in applying mean–variance optimization has been the estimation of stable covariance matrixes and asset weights for large portfolios. Asset returns typically exhibit strong multi-collinearity, making the estimation of the covariance matrix highly sensitive to noise and outliers, so the resulting optimized asset weights are highly unstable. Regularization methods have been used to address this problem.

In prediction, only out-of-sample performance (i.e., prediction accuracy) really matters. The relatively parsimonious models produced by applying penalized regression methods, like LASSO, tend to work well because they are less subject to overfitting.

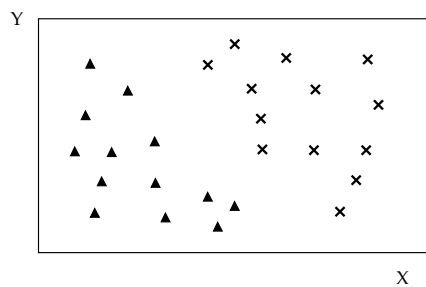
5.2 Support Vector Machine

Support vector machine (SVM) is one of the most popular algorithms in machine learning. It is a powerful supervised algorithm used for classification, regression, and outlier detection. Despite its complicated sounding name, the notion is relatively

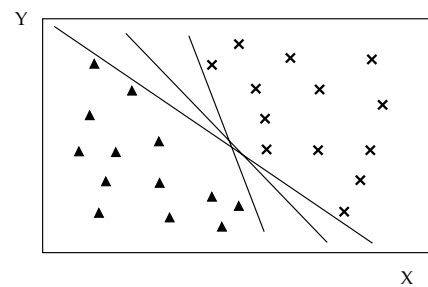
straightforward and best explained with a few pictures. The left panel in Exhibit 7 presents a simple data set with two features (x and y coordinates) labeled in two groups (triangles and crosses). These binary labeled data are noticeably separated into two distinct regions, which could represent stocks with positive and negative returns in a given year. These two regions can be easily separated by numerous straight lines; three of them are shown in the right panel of Exhibit 7. The data are thus linearly separable, and any of the straight lines shown would be called a **linear classifier**—a binary classifier that makes its classification decision based on a linear combination of the features of each data point.

Exhibit 7: Scatterplots and Linear Separation of Labeled Data

A. Data Labeled in Two Groups

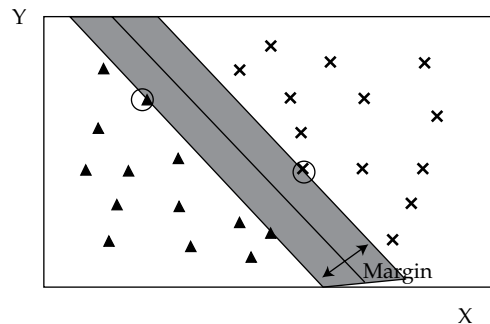


B. Data is Linearly Separable



With two dimensions or features (x and y) linear classifiers can be represented as straight lines. Observations with n features can be represented in an n -dimension space, and the data set would be linearly separable if the observations can be separated into two distinct regions by a linear space boundary. The general term for such a space boundary is an n -dimensional hyperplane, which with $n = 1$ is called a line and with $n = 2$ is called a plane.

Support vector machine (SVM) is a linear classifier that determines the hyperplane that optimally separates the observations into two sets of data points. The intuitive idea behind the SVM algorithm is maximizing the probability of making a correct prediction (here, that an observation is a triangle or a cross) by determining the boundary that is the furthest away from all the observations. In Exhibit 8, SVM separates the data by the maximum margin, where the margin is the shaded strip that divides the observations into two groups. The straight line in the middle of the shaded strip is the discriminant boundary or boundary, for short. We can see that the SVM algorithm produces the widest shaded strip (i.e., the one with the maximum margin on either side of the boundary). The margin is determined by the observations closest to the boundary (the circled points) in each set, and these observations are called support vectors. Adding more training data away from the support vectors will not affect the boundary. In our training data sets, however, the result will be sensitive to the support vectors.

Exhibit 8 Linear Support Vector Machine Classifier

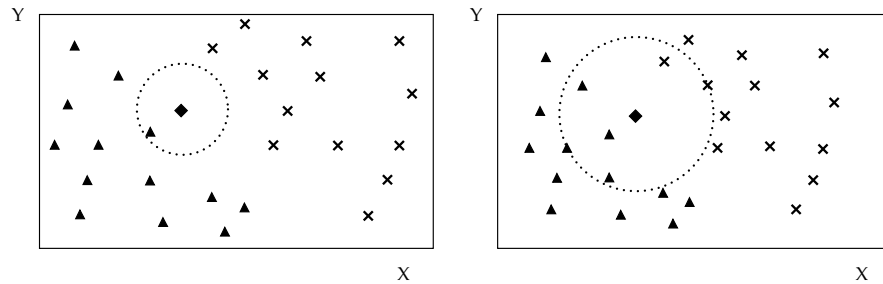
In Exhibit 8, SVM is classifying all observations perfectly. Many real-world data sets, however, are not perfectly linearly separable. Some observations may fall on the wrong side of the boundary and be misclassified by the SVM algorithm. The SVM algorithm handles this problem by an adaptation called **soft margin classification**, which adds a penalty to the objective function for observations in the training set that are misclassified. In essence, the SVM algorithm will choose a discriminant boundary that optimizes the trade-off between a wider margin and a lower total error penalty.

As an alternative to soft margin classification, a non-linear SVM algorithm can be run by introducing more advanced, non-linear separation boundaries. These algorithms will reduce the number of misclassified instances in the training data sets but will have more features, thus adding to the model's complexity.

SVM has many applications in investment management. It is particularly suited for small- to medium-size but complex high-dimensional data sets, such as corporate financial statements or bankruptcy databases. Investors seek to predict company failures for identifying stocks to avoid or to short sell, and SVM can generate a binary classification (e.g., bankruptcy likely vs. bankruptcy unlikely) using many fundamental and technical feature variables. SVM can quickly capture the characteristics of such data with many features while being resilient to outliers and correlated features. SVM can also be used to classify text from documents (e.g., news articles, company announcements, and company annual reports) into useful categories for investors (e.g., positive sentiment and negative sentiment).

5.3 K-Nearest Neighbor

K-nearest neighbor (KNN) is a supervised learning technique most often used for classification and sometimes for regression. The idea is to classify a new observation by finding similarities ("nearness") between this new observation and the existing data. Going back to the scatterplot in Exhibit 7, let us assume we have a new observation: The diamond in Exhibit 9 needs to be classified as belonging to either the cross or the triangle category. If $k = 1$, the diamond will be classified into the same category as its nearest neighbor (i.e., the triangle in the left panel). The right panel in Exhibit 9 presents the case where $k = 5$, so the algorithm will look at the diamond's 5 nearest neighbors, which are 3 triangles and 2 crosses. The decision rule is to choose the classification with the largest number of nearest neighbors out of the 5 being considered. So, the diamond is again classified as belonging to the triangle category.

Exhibit 9 K-Nearest Neighbor (KNN) Algorithm*A. KNN With New Observation, K=1**B. KNN With New Observation, K=5*

Let us suppose we have a database of corporate bonds classified by credit rating that also contains detailed information on the characteristics of these bonds. Such features would include those of the issuing company (e.g., asset size, industry, leverage ratios, cash flow ratios) and of the bond issue itself (e.g., tenor, fixed/floating coupon, embedded options). Now, assume a new bond is about to be issued with no credit rating. By nature, corporate bonds with similar issuer and issue characteristics should be given a similar credit rating. So, by using KNN, we can predict the implied credit rating of the new bond based on the similarities of its characteristics to those of the bonds in our database.

KNN is a straightforward, intuitive model that is still very powerful because it is non-parametric; the model makes no assumptions about the distribution of the data. Moreover, it can be used directly for multi-class classification. A critical challenge of KNN, however, is defining what it means to be “similar” (or near). Besides the selection of features, an important decision relates to the distance metric used to model similarity because an inappropriate measure will generate poorly performing models. The choice of a correct distance measure may be even more subjective for ordinal or categorical data. For example, if an analyst is looking at the similarities in market performance of various equities, he/she may consider using the correlation between the stocks’ historical returns as an appropriate measure of similarity.

Knowledge of the data and understanding of the business objectives of the analysis are critical steps in the process of defining similarity. KNN results can be sensitive to inclusion of irrelevant or correlated features, so it may be necessary to select features manually. By doing so, the analyst removes less-valuable information to keep the most relevant and pertinent information. If done correctly, this process should generate a more representative distance measure. KNN algorithms tend to work better with a small number of features.

Finally, the number k , the hyperparameter of the model, must be chosen with the understanding that different values of k can lead to different conclusions. For predicting the credit rating of an unrated bond, for example, should k be the 3, 15, or 50 most similar bonds to the unrated bond? If k is an even number, there may be ties and no clear classification. Choosing a value for k that is too small would result in a high error rate and sensitivity to local outliers, but choosing a value for k that is too large would dilute the concept of nearest neighbors by averaging too many outcomes. In practice, several different techniques can be used to determine an optimal value for k , taking into account the number of categories and their partitioning of the feature space.

The KNN algorithm has many applications in the investment industry, including bankruptcy prediction, stock price prediction, corporate bond credit rating assignment, and customized equity and bond index creation.

5.4 Classification and Regression Tree

Classification and Regression Tree (CART) is another common supervised machine learning technique that can be applied to predict either a categorical target variable, producing a classification tree, or a continuous target variable, producing a regression tree. Most commonly, CART is applied to binary classification or regression.

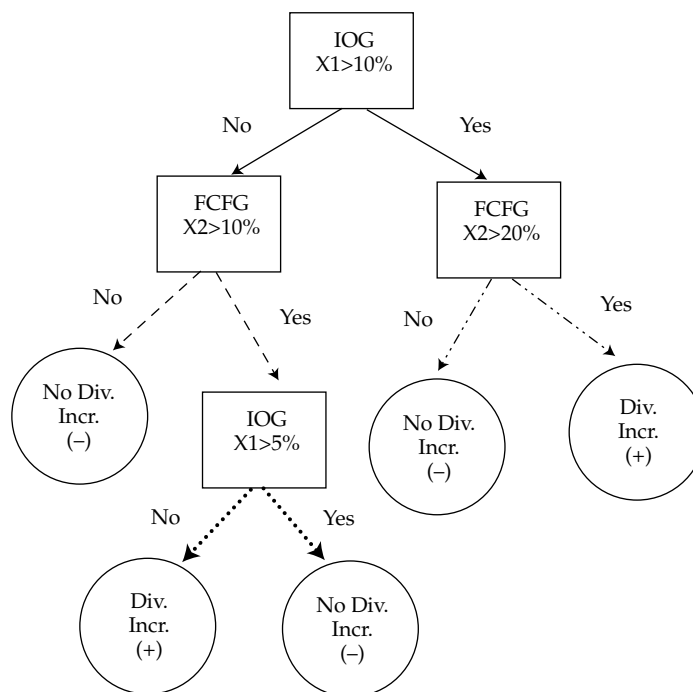
CART will be discussed in the context of a simplified model for classifying companies by whether or not they increase their dividends to shareholders. Such a classification requires a binary tree: a combination of an initial root node, decision nodes, and terminal nodes. The root node and each decision node represent a single feature (f) and a cutoff value (c) for that feature. As shown in Panel A of Exhibit 10, we start at the initial root node for a new data point. In this case, the initial root node represents the feature investment opportunities growth (IOG), designated as X_1 , with a cutoff value of 10%. From the initial root node, the data are partitioned at decision nodes into smaller and smaller subgroups until terminal nodes are formed that contain the predicted labels. In this case, the predicted labels are either dividend increase (the cross) or no dividend increase (the dash).

Also shown in Panel A of Exhibit 10, if the value of feature IOG (X_1) is greater than 10% (Yes), then we proceed to the decision node for free cash flow growth (FCFG), designated as X_2 , which has a cutoff value of 20%. Now, if the value of FCFG is not greater than 20% (No), then CART will predict that data point belongs to the no dividend increase (dash) category, which represents a terminal node. Conversely, if the value of X_2 is greater than 20% (Yes), then CART will predict that data point belongs to the dividend increase (cross) category, which represents another terminal node.

It is important to note that the same feature can appear several times in a tree in combination with other features. Moreover, some features may be relevant only if other conditions have been met. For example, going back to the initial root node, if IOG is not greater than 10% ($X_1 \leq 10\%$) and FCFG is greater than 10%, then IOG appears again as another decision node, but this time it is lower down in the tree and has a cutoff value of 5%.

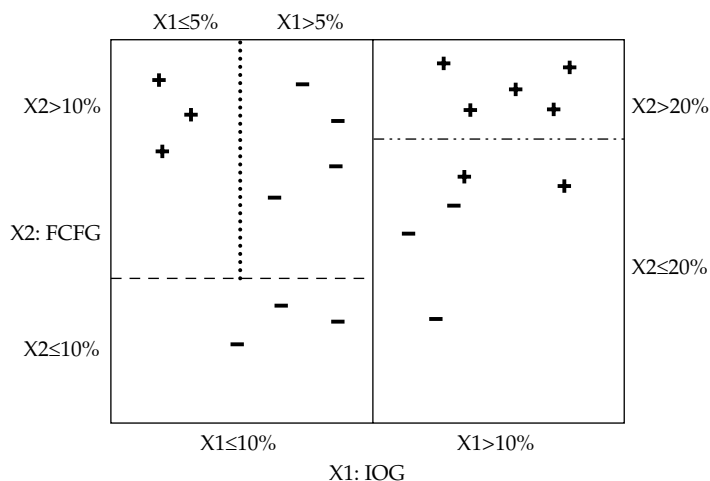
Exhibit 10 Classification and Regression Tree—Decision Tree and Partitioning of the Feature Space

A. Decision Tree



X1: Investment Opportunities Growth (IOG)
X2: Free Cashflow Growth (FCFG)

B. Partitioning of the Feature (X1,X2) Feature Space



We now turn to how the CART algorithm selects features and cutoff values for them. Initially, the classification model is trained from the labeled data, which in this hypothetical case are 10 instances of companies having dividend increase (the crosses) and 10 instances of companies with no dividend increase (the dashes). As shown in Panel B of Exhibit 10, at the initial root node and at each decision node the feature space (i.e., the plane defined by X1 and X2) is split into two rectangles for values above and below the cutoff value for the particular feature represented at that

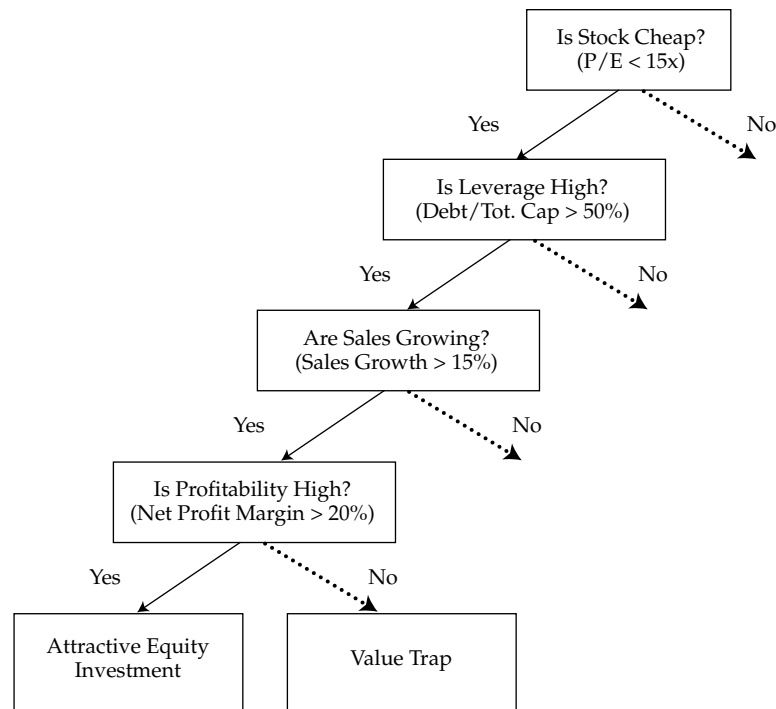
node. This can be seen by noting the distinct patterns of the lines that emanate from the decision nodes in Panel A. These same distinct patterns are used for partitioning the feature space in Panel B.

The CART algorithm chooses the feature and the cutoff value at each node that generates the widest separation of the labeled data to minimize classification error (e.g., by a criterion, such as mean-squared error). After each decision node, the partition of the feature space becomes smaller and smaller, so observations in each group have lower within-group error than before. At any level of the tree, when the classification error does not diminish much more from another split (bifurcation), the process stops, the node is a terminal node, and the category that is in the majority at that node is assigned to it. If the objective of the model is classification, then the prediction of the algorithm at each terminal node will be the category with the majority of data points. For example, in Panel B of Exhibit 10, the top right rectangle of the feature space, representing $\text{IOG } (X1) > 10\%$ and $\text{FCFG } (X2) > 20\%$, contains 5 crosses, the most data points of any of the partitions. So, CART would predict that a new data point (i.e., a company) with such features belongs to the cross (dividend increase) category. However, if instead the new data point had $\text{IOG } (X1) > 10\%$ and $\text{FCFG } (X2) \leq 20\%$, then it would be predicted to belong to the dash (no dividend increase) category—represented by the lower right rectangle with 2 crosses but with 3 dashes. Finally, if the goal is regression, then the prediction at each terminal node is the mean of the labeled values.

CART makes no assumptions about the characteristics of the training data, so if left unconstrained, potentially it can perfectly learn the training data. To avoid such overfitting, regularization parameters can be added, such as the maximum depth of the tree, the minimum population at a node, or the maximum number of decision nodes. The iterative process of building nodes is stopped once the regularization criterion has been reached. For example, in Panel B of Exhibit 10, the upper left rectangle of the feature space (determined by $X1 \leq 10\%$, $X2 > 10\%$, and $X1 \leq 5\%$ with three crosses) might represent a terminal node resulting from a regularization criterion with minimum population equal to 3. Alternatively, regularization can occur via a **pruning** technique that can be used afterward to reduce the size of the tree. Sections of the tree that provide little classifying power are pruned (i.e., removed).

By its iterative structure, CART can uncover complex dependencies between features that other models cannot reveal. As demonstrated in Exhibit 10, the same feature can appear several times in combination with other features and some features may be relevant only if other conditions have been met.

As shown in Exhibit 11, high profitability is a critical feature for predicting if a stock is an attractive investment or a value trap (i.e., an investment that, although apparently priced cheaply, is likely to be unprofitable). This feature is relevant only if the stock is cheap—for example, in this hypothetical case if P/E is less than 15, leverage is high (debt to total capital > 50%), and sales are expanding (sales growth > 15%). Said another way, high profitability is irrelevant in this context if the stock is not cheap, *and* if leverage is not high, *and* if sales are not expanding. Multiple linear regression typically fails in such situations where the relationship between the features and the outcome is non-linear.

Exhibit 11 Stylized Decision Tree—Attractive Investment or Value Trap?

CART is a popular supervised machine learning model because the tree provides a visual explanation for the prediction. This contrasts favorably with other algorithms that are often considered to be “black boxes” because it may be difficult to understand the reasoning behind their outcomes and thus to foster trust in them. CART is a powerful tool to build expert systems for decision-making processes. It can induce robust rules despite noisy data and complex relationships between high numbers of features. Typical applications of CART in investment management include, among others, enhancing detection of fraud in financial statements, generating consistent decision processes in equity and fixed-income selection, and simplifying communication of investment strategies to clients.

5.5 Ensemble Learning and Random Forest

Instead of basing predictions on the results of a single model as in the previous discussion, why not use the predictions of a group—or an ensemble—of models? Each single model will have a certain error rate and will make noisy predictions. But by taking the average result of many predictions from many models, we can expect to achieve a reduction in noise as the average result converges toward a more accurate prediction. This technique of combining the predictions from a collection of models is called **ensemble learning**, and the combination of multiple learning algorithms is known as the **ensemble method**. Ensemble learning typically produces more accurate and more stable predictions than the best single model. In fact, in many prestigious machine learning competitions, an ensemble method is often the winning solution.

Ensemble learning can be divided into two main categories: 1) an ensemble method can be an aggregation of heterogeneous learners (i.e., different types of algorithms combined together with a voting classifier); or 2) an ensemble method can be an

aggregation of homogenous learners (i.e., a combination of the same algorithm, using different training data that are based, for example, on a bootstrap aggregating (i.e., bagging) technique as discussed later).

5.5.1 Voting Classifiers

Suppose you have been working on a machine learning project for some time and have trained and compared the results of several algorithms, such as SVM, KNN, and CART. A **majority-vote classifier** will assign to a new data point the predicted label with the most votes. For example, if the SVM and KNN models are both predicting the category “stock outperformance” and the CART model is predicting the category “stock underperformance,” then the majority-vote classifier will choose “stock outperformance.” The more individual models you have trained, the higher the accuracy of the aggregated prediction up to a point. There is an optimal number of models beyond which performance would be expected to deteriorate from overfitting. The trick is to look for diversity in the choice of algorithms, modelling techniques, and hypotheses. The (extreme) assumption here is that if the predictions of the individual models are independent, then we can use the law of large numbers to achieve a more accurate prediction.

5.5.2 Bootstrap Aggregating (Bagging)

Alternatively, one can use the same machine learning algorithm but with different training data. **Bootstrap aggregating (or bagging)** is a technique whereby the original training data set is used to generate n new training data sets or bags of data. Each new bag of data is generated by random sampling with replacement from the initial training set. The algorithm can now be trained on n independent data sets that will generate n new models. Then, for each new observation, we can aggregate the n predictions using a majority-vote classifier for a classification or an average for a regression. Bagging is a very useful technique because it helps to improve the stability of predictions and protects against overfitting the model.

5.5.3 Random Forest

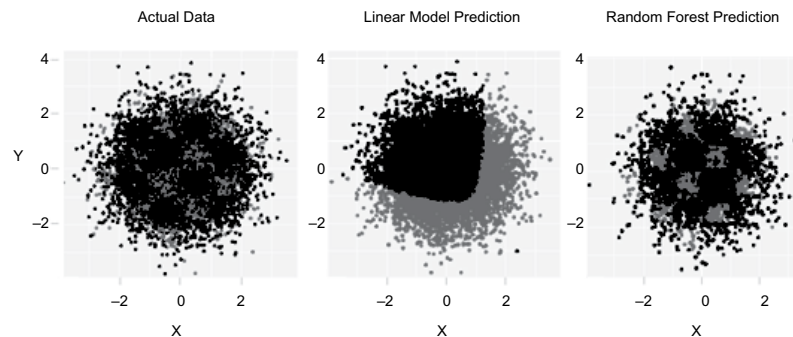
A **random forest classifier** is a collection of a large number of decision trees trained via a bagging method. For example, a CART algorithm would be trained using each of the n independent data sets (from the bagging process) to generate the multitude of different decision trees that make up the random forest classifier.

To derive even more individual predictions, added diversity can be generated in the trees by randomly reducing the number of features available during training. So, if each observation has n features, one can randomly select a subset of m features (where $m < n$) that will then be considered by the CART algorithm for splitting the data set at each of the decision nodes. The number of subset features (m), the number of trees to use, the minimum size (population) of each node (or leaf), and the maximum depth of each tree are all hyperparameters that can be tuned to improve overall model prediction accuracy. For any new observation, we let all the classifier trees (the “random forest”) undertake classification by majority vote—implementing a machine learning version of the “wisdom of crowds.” The process involved in random forest tends to protect against overfitting on the training data. It also reduces the ratio of noise to signal because errors cancel out across the collection of slightly different classification trees. However, an important drawback of random forest is that it lacks the ease of interpretability of individual trees; as a result, it is considered a relatively black box-type algorithm.

Exhibit 12 presents three scatterplots of actual and predicted defaults by small- and medium-sized businesses with respect to two features, X and Y—for example, firm profitability and leverage, respectively. The left plot shows the actual cases of default

in light shade and no default in dark shade, while the middle and right plots present the predicted defaults and no defaults (also in light and dark shades, respectively). It is clear from the middle plot, which is based on a traditional linear regression model, that the model fails to predict the complex non-linear relationship between the features. Conversely, the right plot, which presents the prediction results of a random forest model, shows that this model performs very well in matching the actual distribution of the data.

Exhibit 12 Credit Defaults of Small- and Medium-Sized Borrowers



Source: Bacham and Zhao (2017).

Ensemble Learning with Random Forest

In making use of voting across classifier trees, random forest is an example of ensemble learning: Incorporating the output of a collection of models produces classifications that have better signal-to-noise ratios than the individual classifiers. A good example of a credit card fraud detection problem comes from an open source data set on Kaggle.¹ Here, the data contained several anonymized features that might be used to explain which transactions were fraudulent. The difficulty in the analysis arises from the fact that the rate of fraudulent transactions is very low; in a sample of 284,807 transactions, only 492 were fraudulent (0.17%). This is akin to finding a needle in a haystack. Applying a random forest classification algorithm with an oversampling technique—which involves increasing the proportional representation of fraudulent data in the training set—does extremely well. Despite the lopsided sample, it delivers precision (the percentage of cases marked fraudulent that were actually fraudulent) of 89% and recall (the percentage of fraudulent cases that were marked as fraudulent) of 82%.

Despite its relative simplicity, random forest is a powerful algorithm with many investment applications. These include, for example, use in factor-based investment strategies for asset allocation and investment selection or use in predicting whether an IPO will be successful (e.g., percent oversubscribed, first trading day close/IPO price) given the attributes of the IPO offering and the corporate issuer.

¹ See <https://www.kaggle.com/mlg-ulb/creditcardfraud> (accessed 1 October 2018).

EXAMPLE 3**Support Vector Machine and K-Nearest Neighbor**

Rachel Lee is a fixed-income portfolio manager analyst with Zeta Investment Management Company. Zeta manages an investment-grade bond portfolio for small, conservative institutions and a non-investment-grade (i.e., high-yield) bond portfolio for yield-seeking, high-net-worth individuals. Both portfolios can hold unrated bonds if the characteristics of the unrated bonds closely match those of the respective portfolio's average holding.

Lee is discussing an upcoming straight, 10-year fixed coupon bond issue with senior credit analyst Marc Watson. Watson comments that although the bond's issuer, Biotron Corporation, has not had this issue rated, his analysis of the company's profitability, cash flow, leverage, and coverage ratios places the issue near the borderline between low investment-grade (Baa3/BBB-) and high non-investment-grade (Ba1/BB+) bonds.

Lee decides to use machine learning methods to confirm the implied credit rating of Biotron Corporation.

- 1 State the type of problem being addressed by Lee.
- 2 State two ML algorithms that Lee could use to explore the implied credit rating of Biotron Corporation, and then describe how each algorithm could be applied.

Lee decides to apply the two identified ML algorithms. Both algorithms clearly support a high non-investment-grade rating. Watson states that because both ML algorithms agree on the rating, he has confidence in relying on the rating.

- 3 State one argument in support of Watson's viewpoint.

Solution to 1:

Lee is addressing a supervised learning classification problem because she must determine whether Biotron's upcoming bond issue would be classified as investment grade or non-investment grade.

Solution to 2:

One suitable ML algorithm is the SVM. The SVM algorithm is a linear classifier that aims to seek the optimal hyperplane—the one that separates observations into two distinct sets by the maximum margin. So, the SVM is well suited to binary classification problems, such as the one facing Lee (investment grade vs. non-investment grade). In this case, Lee could train the SVM algorithm on data—characteristics (features) and rating (target)—of low investment-grade (Baa3/BBB-) and high non-investment-grade (Ba1/BB+) bonds. Lee would then note on which side of the margin the new data point (Biotron's new bonds) lies.

The KNN algorithm is also well suited for classification problems because it classifies a new observation by finding similarities (or nearness) between the new observation and the existing data. Training the algorithm with data as for SVM, the decision rule for classifying Biotron's new bonds is which classification is in the majority among its k -nearest neighbors. Note that k must be pre-specified by Lee.

Solution to 3:

If the ML algorithms disagreed on the classification, the classification would be more likely to be sensitive to the algorithm's approach to classifying data. Because the classification of Biotron's new issue appears robust to the choice of ML algorithm (i.e., both algorithms agree on the rating), the resulting classification will likely be correct.

EXAMPLE 4**CART and Ensemble Learning**

Laurie Kim is a portfolio manager at Hilux LLC, a high-yield bond investment firm. The economy has been in recession for several months, and high-yield bond prices have declined precipitously as credit spreads have widened in response to the weak macroeconomic environment. Kim, however, believes this is a good time to buy as she expects to profit as credit spreads narrow and high-yield bond prices rise in anticipation of economic recovery.

Based on her analysis, Kim believes that corporate high-yield bonds in the credit quality range of B/B2 to CCC/Caa2 are the most attractive. However, she must carefully select which bonds to buy and which bonds to avoid because of the elevated default risk caused by the currently weak economy.

To help with her bond selection, Kim turns to Hilux's data analytics team. Kim has supplied them with historical data consisting of 19 fundamental and 5 technical factors for several thousand high-yield bond issuers and issues labeled to indicate default or no default. Kim requests that the team develop an ML-based model that will make accurate classifications in two categories: default and no default. Exploratory data analysis suggests considerable non-linearities among the feature set.

- 1 State the type of problem being addressed by Kim.
- 2 Describe the dimensionality of the model that Kim requests her analytics team to develop.
- 3 Evaluate whether a CART model is appropriate for addressing her problem.
- 4 Describe how a CART model operates at each node of the tree.
- 5 Describe how the team might avoid overfitting and improve the predictive power of a CART model.
- 6 Describe how ensemble learning might be used by the team to develop even better predictions for Kim's selection of corporate high-yield bonds.

Solution to 1:

Kim is addressing a classification problem because she must determine whether bonds that she is considering purchasing in the credit quality range of B/B2 to CCC/Caa2 will default or not default.

Solution to 2:

With 19 fundamental and 5 technical factors (i.e., the features) the dimensionality of the model is 24.

Solution to 3:

The CART model is an available algorithm for addressing classification problems. Its ability to handle complex, non-linear relationships makes it a good choice to address the modelling problem at hand. An important advantage of CART is that its results are relatively straightforward to visualize and interpret, which should help Kim explain her recommendations based on the model to Hilux's investment committee and the firm's clients.

Solution to 4:

At each node in the decision tree, the algorithm will choose the feature and the cutoff value for the selected feature that generates the widest separation of the labeled data to minimize classification error.

Solution to 5:

The team can avoid overfitting and improve the predictive power of the CART model by adding regularization parameters. For example, the team could specify the maximum depth of the tree, the minimum population at a node, or the maximum number of decision nodes. The iterative process of building nodes will be stopped once the regularization criterion has been reached. Alternatively, a pruning technique can be used afterward to remove parts of the CART model that provide little power to correctly classify instances into default or no default categories.

Solution to 6:

The analytics team might use ensemble learning to combine the predictions from a collection of models, where the average result of many predictions leads to a reduction in noise and thus more accurate predictions. Ensemble learning can be achieved by an aggregation of either heterogeneous learners—different types of algorithms combined with a voting classifier—or homogenous learners—a combination of the same algorithm but using different training data based on the bootstrap aggregating (i.e., bagging) technique. The team may also consider developing a random forest classifier (i.e., a collection of many decision trees) trained via a bagging method.

UNSUPERVISED MACHINE LEARNING ALGORITHMS

6

As you may recall, unsupervised learning is machine learning that does not use labeled data (i.e., no target variable); thus, the algorithms are tasked with finding patterns within the data themselves. The two main types of unsupervised ML algorithms shown in Exhibits 2 and 3 are dimension reduction, using principal components analysis, and clustering, which includes k-means and hierarchical clustering. These will now be described in turn.

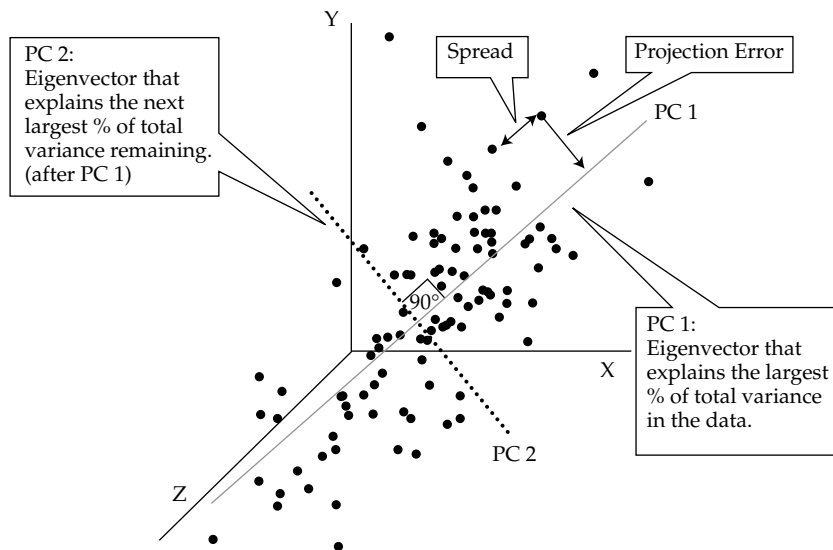
6.1 Principal Components Analysis

Dimension reduction is an important type of unsupervised learning that is used widely in practice. When many features are in a data set, representing the data visually or fitting models to the data may become extremely complex and “noisy” in the sense of reflecting random influences specific to a data set. In such cases, dimension reduction may be necessary. Dimension reduction aims to represent a data set with many, typically correlated, features by a smaller set of features that still do well in describing the data.

A long-established statistical method for dimension reduction is principal components analysis (PCA). PCA is used to summarize or reduce highly correlated features of data into a few main, uncorrelated composite variables. A **composite variable** is a variable that combines two or more variables that are statistically strongly related to each other. Informally, PCA involves transforming the covariance matrix of the features and involves two key concepts: eigenvectors and eigenvalues. The **eigenvectors** define new, mutually uncorrelated composite variables that are linear combinations of the original features. As a vector, an eigenvector also represents a direction. Associated with each eigenvector is an eigenvalue. An **eigenvalue** gives the proportion of total variance in the initial data that is explained by each eigenvector. The PCA algorithm orders the eigenvectors from highest to lowest according to their eigenvalues—that is, in terms of their usefulness in explaining the total variance in the initial data. PCA selects as the first principal component the eigenvector that explains the largest proportion of variation in the data set (the eigenvector with the largest eigenvalue). The second principal component explains the next largest proportion of variation remaining after the first principal component; it continues for the third, fourth, and subsequent principal components. As the principal components are linear combinations of the initial feature set, only a few principal components are typically required to explain most of the total variance in the initial feature covariance matrix.

Exhibit 13 shows a hypothetical data set with three features, so it is plotted in three dimensions along the x, y, and z axes. Each data point has a measurement (x, y, z), and the data should be standardized so that the mean of each series (x's, y's, and z's) is 0 and the standard deviation is 1. Assume PCA has been applied, revealing the first two principal components, PC1 and PC2. With respect to PC1, a perpendicular line dropped from each data point to PC1 shows the vertical distance between the data point and PC1, representing **projection error**. Moreover, the distance between each data point in the direction that is parallel to PC1 represents the spread or variation of the data along PC1. The PCA algorithm operates in such a way that it finds PC1 by selecting the line for which the sum of the projection errors for all data points is *minimized* and for which the sum of the spread between all the data is *maximized*. As a consequence of these selection criteria, PC1 is the unique vector that accounts for the largest proportion of the variance in the initial data. The next largest portion of the remaining variance is best explained by PC2, which is at right angles to PC1 and thus is uncorrelated with PC1. The data points can now be represented by the first two principal components. This example demonstrates the effectiveness of the PCA algorithm in summarizing the variability of the data and the resulting dimension reduction.

Exhibit 13 First and Second Principal Components of a Hypothetical 3-Dimensional Data Set



It is important to know how many principal components to retain because there is a trade-off between a lower-dimensional, more manageable view of a complex data set when a few are selected and some loss of information. **Scree plots**, which show the proportion of total variance in the data explained by each principal component, can be helpful in this regard (see the accompanying sidebar). In practice, the smallest number of principal components that should be retained is that which the scree plot shows as explaining 85% to 95% of total variance in the initial data set.

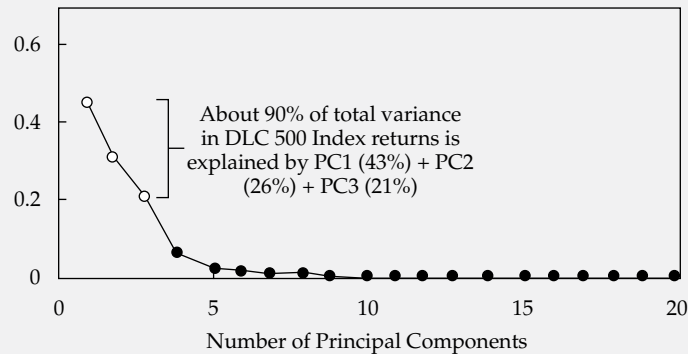
Scree Plots for the Principal Components of Returns to the Hypothetical DLC 500 and VLC 30 Equity Indexes

In this illustration, researchers use scree plots and decide that three principal components are sufficient for explaining the returns to the hypothetical DLC 500 and VLC 30 equity indexes over the last 10-year period. The DLC 500 can be thought of as a diversified index of large-cap companies covering all economic sectors, while the VLC 30 is a more concentrated index of the 30 largest publicly traded companies. The data set consists of index prices and more than 2,000 fundamental and technical features. Multi-collinearity among the features is a typical problem because that many features or combinations of features would tend to have overlaps. To mitigate the problem, PCA can be used to capture the information and variance in the data. The following scree plots show that of the 20 principal components generated, the first 3 together explain about 90% and 86% of the variance in the value of the DLC 500 and VLC 30 indexes, respectively. The scree plots indicate that for each of these indexes, the incremental contribution to explaining the variance structure of the data is quite small after about the fifth principal component. Therefore, these less useful principal components can be ignored without much loss of information.

Scree Plots of Percent of Total Variance Explained by Each Principal Component for Hypothetical DLC 500 and VLC 30 Equity Indexes

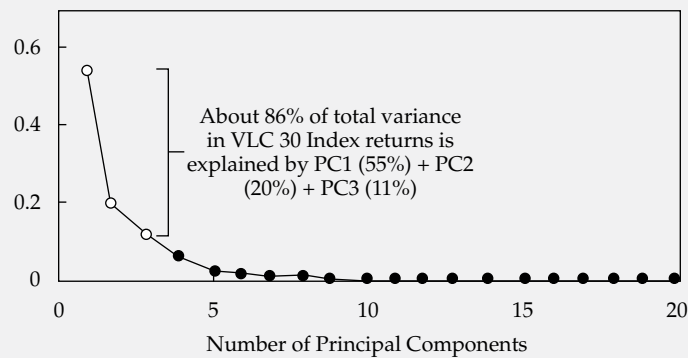
A. Diversified Large Cap 500 Index (DLC 500)

Percent of Variance Explained



B. Very Large Cap 30 Index (VLC 30)

Percent of Variance Explained



The main drawback of PCA is that since the principal components are combinations of the data set's initial features, they typically cannot be easily labeled or directly interpreted by the analyst. Compared to modelling data with variables that represent well-defined concepts, the end user of PCA may perceive PCA as something of a "black box."

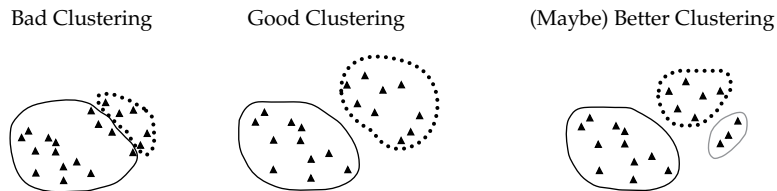
Reducing the number of features to the most relevant predictors is very useful, even when working with data sets having as few as ten or so features. Notably, dimension reduction facilitates visually representing the data in two or three dimensions. It is typically performed as part of exploratory data analysis, before training another supervised or unsupervised learning model. Machine learning models are quicker to train, tend to reduce overfitting (by avoiding the curse of dimensionality), and are easier to interpret if provided with lower dimensional data sets.

6.2 Clustering

Clustering is another type of unsupervised machine learning that is used to organize data points into similar groups called clusters. A **cluster** contains a subset of observations from the data set such that all the observations within the same cluster are deemed "similar." The aim is to find a good clustering of the data—meaning that the observations inside each cluster are similar or close to each other (a property known

as cohesion) and the observations in two different clusters are as far away from one another or are as dissimilar as possible (a property known as separation). Exhibit 14 depicts this intra-cluster cohesion and inter-cluster separation.

Exhibit 14 Evaluating Clustering—Intra-Cluster Cohesion and Inter-Cluster Separation



Clustering algorithms are particularly useful in the many investment problems and applications in which the concept of similarity is important. Applied to grouping companies, for example, clustering may uncover important similarities and differences among companies that are not captured by standard classifications of companies by industry and sector. In portfolio management, clustering methods have been used for improving portfolio diversification.

In practice, expert human judgment has a role in using clustering algorithms. In the first place, one must establish what it means to be “similar.” Each company can be considered an observation with multiple features, including such financial statement items as total revenue and profit to shareholders, a wide array of financial ratios, or any other potential model inputs. Based on these features, a measure of similarity or “distance” between two observations (i.e., companies) can be defined. The smaller the distance, the more similar the observations; the larger the distance, the more dissimilar the observations.

A commonly used definition of distance is the Euclidian distance, the straight-line distance between two points. Roughly a dozen different distance measures are used regularly in ML. In practice, the choice of the distance measures depends on the nature of the data (numerical or not) and the business problem being investigated. Once the relevant distance measure is defined, similar observations can be grouped together. We now introduce two of the more popular clustering approaches: k-means and hierarchical clustering.

6.2.1 K-Means Clustering

K-means is a relatively old algorithm that repeatedly partitions observations into a fixed number, k , of non-overlapping clusters. The number of clusters, k , is a model hyperparameter—a parameter whose value must be set by the researcher before learning begins. Each cluster is characterized by its **centroid** (i.e., center), and each observation is assigned by the algorithm to the cluster with the centroid to which that observation is closest.

The k-means algorithm follows an iterative process. It is illustrated in Exhibit 15 for $k = 3$ and a set of observations on a variable that can be described by two features. In Exhibit 15, the horizontal and vertical axes represent, respectively, the first and second characteristics. For example, an investment analyst may want to group a set of firms into three groups according to two numerical measures of management quality. The algorithm groups the observations in the following steps:

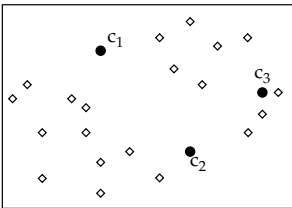
- 1 K-means starts by determining the position of the k (here, 3) initial random centroids.

- 2 The algorithm then analyzes the features for each observation. Based on the distance measure that is utilized, k-means assigns each observation to its closest centroid, which defines a cluster.
- 3 Using the observations within each cluster, k-means then calculates the new (k) centroids for each cluster, where the centroid is the average value of their assigned observations.
- 4 K-means then reassigns the observations to the new centroids, redefining the clusters in terms of included and excluded observations.
- 5 The process of recalculating the new (k) centroids for each cluster is reiterated.
- 6 K-means then reassigns the observations to the revised centroids, again redefining the clusters in terms of observations that are included and excluded.

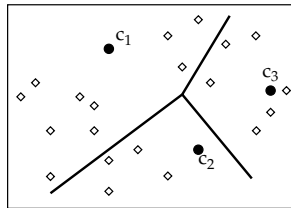
The k-means algorithm will continue to iterate until no observation is reassigned to a new cluster (i.e., no need to recalculate new centroids). The algorithm has then converged and reveals the final k clusters with their member observations. The k-means algorithm has minimized intra-cluster distance (thereby maximizing cohesion) and has maximized inter-cluster distance (thereby maximizing separation) under the constraint that $k = 3$.

Exhibit 15 Example of 3-Means Algorithm

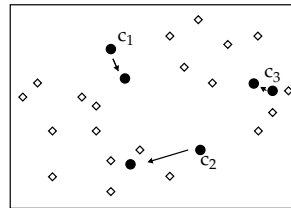
Step 1: Chooses initial random centroids: c_1, c_2, c_3



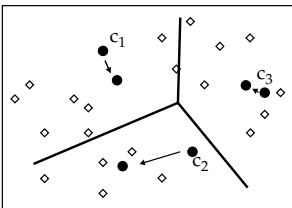
Step 2: Assigns each observation to nearest centroid (defining initial 3 clusters)



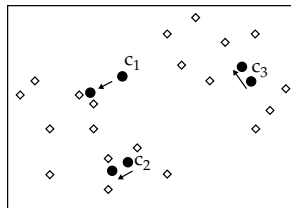
Step 3: Calculates new centroids as the average values of observations in a cluster



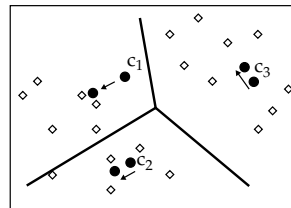
Step 4: Reassigns each observation to the nearest centroid (from Step 3)



Step 5: Reiterates the process of recalculating new centroids



Step 6: Reassigns each observation to the nearest centroid (from Step 5), completing second iteration



The k-means algorithm is fast and works well on very large data sets with hundreds of millions of observations. However, the final assignment of observations to clusters can depend on the initial location of the centroids. To address this problem, the algorithm can be run several times using different sets of initial centroids, and then one can choose the clustering that is most useful given the business purpose.

One limitation of this technique is that the hyperparameter, k , the number of clusters in which to partition the data, must be decided before k-means can be run. So, one needs to have a sense of how many clusters are reasonable for the problem

under investigation and the data set being analyzed. Alternatively, one can run the algorithm using a range of values for k to find the optimal number of clusters—the k that minimizes intra-cluster distance and thereby maximizes intra-cluster similarity (i.e., cohesion) and that maximizes inter-cluster distance (i.e., separation). However, note that the final results can be subjective and dependent on the context of the problem and the particular training set.

For example, consider the Russell 3000 Index, which tracks the 3,000 highest market capitalization stocks in the United States. These 3,000 stocks can be grouped in 10, 50, or even more clusters based on their financial characteristics (e.g., total assets, total revenue, profitability, leverage) and operating characteristics (e.g., employee headcount, R&D intensity). As companies in the same standard industry classification can have very different financial and operating characteristics, using k-means to derive different clusters can provide insights and understanding into the nature of “peer” groups. As mentioned, the exact choice of the k , the number of clusters, will depend on the level of precision or segmentation desired. In a similar vein, clustering can be used to classify collective investment vehicles or hedge funds as an alternative to standard classifications. Clustering analysis can also help visualize the data and facilitate detecting trends or outliers.

In sum, the k-means algorithm is among the most used algorithms in investment practice, particularly in data exploration for discovering patterns in high dimensional data or as a method for deriving alternatives to existing static industry classifications.

6.2.2 Hierarchical Clustering: Agglomerative and Divisive

Hierarchical clustering is an iterative procedure used to build a hierarchy of clusters. In k-means clustering, the algorithm segments the data into a predetermined number of clusters; there is no defined relationship among the resulting clusters. In hierarchical clustering, however, the algorithms create intermediate rounds of clusters of increasing (in “agglomerative”) or decreasing (in “divisive”) size until a final clustering is reached. The process creates relationships among the rounds of clusters, as the word “hierarchical” suggests. Although more computationally intensive than k-means clustering, hierarchical clustering has the advantage of allowing the investment analyst to examine alternative segmentations of data of different granularity before deciding which one to use.

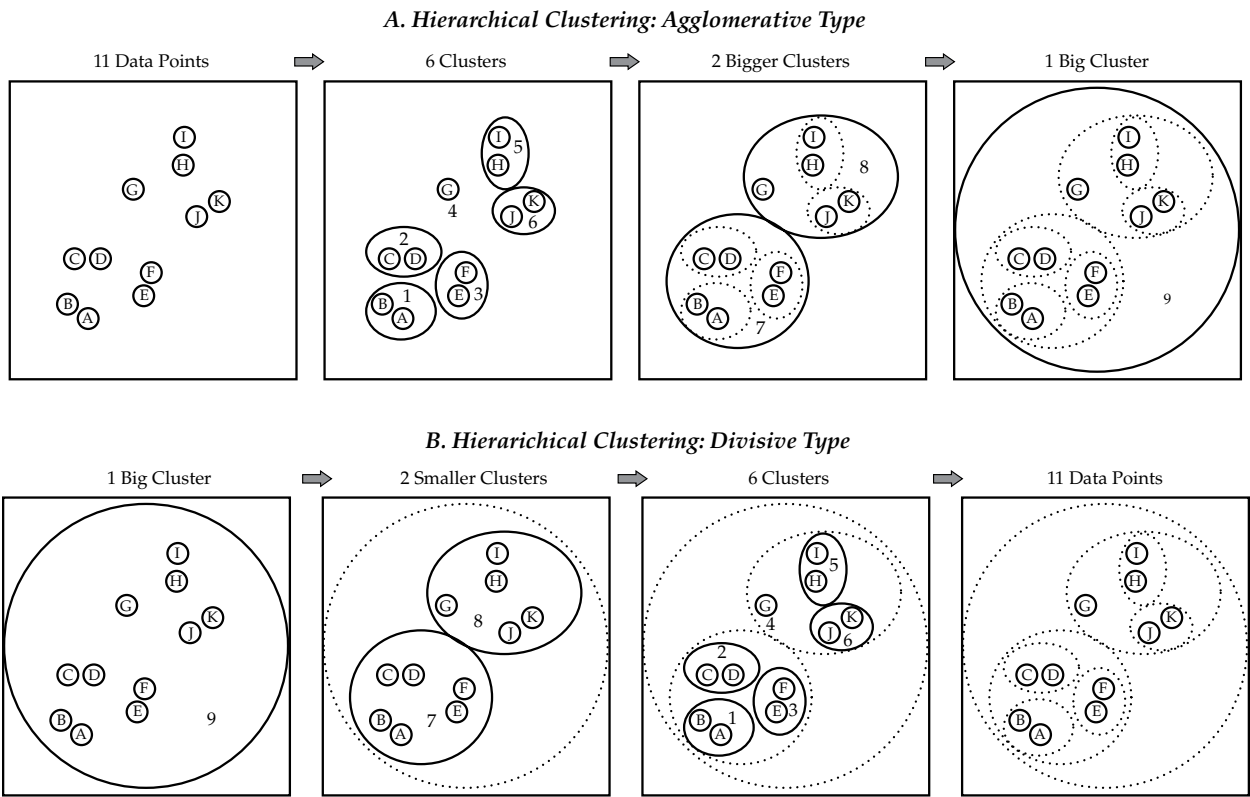
In more detail, **agglomerative clustering** (or bottom-up) hierarchical clustering begins with each observation being treated as its own cluster. Then, the algorithm finds the two closest clusters, defined by some measure of distance (similarity), and combines them into one new larger cluster. This process is repeated iteratively until all observations are clumped into a single cluster. A hypothetical example of how agglomerative clustering develops a hierarchical clustering scheme is depicted in the top part of Exhibit 16, where observations are lettered (A to K) and circles around observations denote clusters. The process begins with 11 individual clusters and then generates a sequence of groupings. The first sequence includes 5 clusters with 2 observations each and 1 cluster with a single observation, G, for a total of 6 clusters. It then generates 2 clusters, 1 cluster with 6 observations and the other with 5 observations. The final result is 1 large cluster containing all 11 observations. It is easily seen that this final large cluster includes the two main sub-clusters, with each containing three smaller sub-clusters.

By contrast, **divisive clustering** (or top-down) hierarchical clustering starts with all the observations belonging to a single cluster. The observations are then divided into two clusters based on some measure of distance (similarity). The algorithm then progressively partitions the intermediate clusters into smaller clusters until each cluster contains only 1 observation. Divisive clustering is depicted in the bottom part of Exhibit 16, which begins with all 11 observations in 1 large cluster. Next, the algorithm generates 2 smaller clusters, one with 6 observations and the other with

5 observations; then 6 clusters, with 2 observations each except for observation G, which is its own cluster. Finally, 11 clusters are generated, with each cluster containing only 1 observation.

In this hypothetical illustration, but not typically (as the two methods generally use different algorithms), the agglomerative and divisive clustering methods produced the same result: two main sub-clusters each having three smaller sub-clusters. The analyst could decide between using a 6- or a 2-cluster representation of the data. The agglomerative method is the approach typically used with large data sets because of the algorithm’s fast computing speed. The agglomerative clustering algorithm makes clustering decisions based on local patterns without initially accounting for the global structure of the data. As such, the agglomerative method is well suited for identifying small clusters. However, as the divisive method starts with a holistic representation of the data, the divisive clustering algorithm is designed to account for the global structure of the data and thus is better suited for identifying large clusters.

Exhibit 16 Agglomerative and Divisive Hierarchical Clustering



To decide on the closest clusters for combining in the agglomerative process or for dividing in the divisive process, an explicit definition for the distance between two clusters is required. Some commonly used definitions for the distance between two clusters involve finding the minimum, the maximum, or the average of the straight-line distances between all the pairs of observations in each cluster.

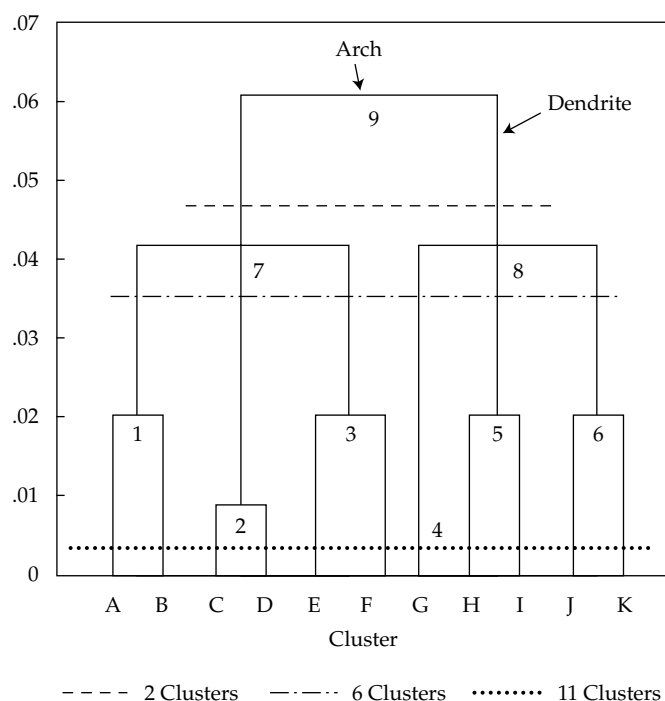
6.2.3 Dendrograms

A type of tree diagram for visualizing a hierarchical cluster analysis known as a **dendrogram** highlights the hierarchical relationships among the clusters. Exhibit 17 shows a dendrogram representation for the clustering shown in Exhibit 16. First, a

few technical points on dendrograms bear mentioning—although they may not all be apparent in Exhibit 17. The x-axis shows the clusters, and the y-axis indicates some distance measure. Clusters are represented by a horizontal line, the arch, which connects two vertical lines, called dendrites, where the height of each arch represents the distance between the two clusters being considered. Shorter dendrites represent a shorter distance (and greater similarity) between clusters. The horizontal dashed lines cutting across the dendrites show the number of clusters into which the data are split at each stage.

The agglomerative algorithm starts at the bottom of the dendrite where each observation is its own cluster (A to K). Agglomerative clustering then generates the 6 larger clusters (1 to 6). For example, clusters A and B combine to form cluster 1, and observation G remains its own cluster, now cluster 4. Moving up the dendrogram, 2 larger clusters are formed, where, for example, cluster 7 includes clusters 1 to 3. Finally, at the top of the dendrogram is the single large cluster (9). The dendrogram readily shows how this largest cluster is composed of the two main sub-clusters (7 and 8), each having three smaller sub-clusters (1 to 3 and 4 to 6, respectively). The dendrogram also facilitates visualization of divisive clustering by starting at the top of the largest cluster and then working downward until the bottom is reached where all 11 single-observation clusters are shown.

Exhibit 17 Dendrogram of Agglomerative Hierarchical Clustering



Clustering has many applications in investment management. For example, portfolio diversification can be approached as a clustering problem with the aim of optimally diversifying risks by investing in assets from multiple different clusters. Because the clusters have maximum inter-cluster separation, diversifying among them helps ensure that the portfolio reflects a wide diversity of characteristics with well-diversified risk. Alternatively, information that investments are concentrated in a cluster indicates a high probability of concentrated risk. Finally, it is important to note that while the results of clustering algorithms are often difficult to evaluate (because the resulting

clusters themselves are not explicitly defined), they are still very useful in practice for uncovering important underlying structure (namely, similarities among observations) in complex data sets.

EXAMPLE 5

Investment Uses of Clustering Algorithms

István Perényi is a portfolio manager of the Europe Diversified Equity Fund (“the Fund”) within the Diversified Investment Management Company (DIMCO) fund family. The Fund is benchmarked to the STOXX Europe 600 Index, which spans 17 countries, 19 industry sectors, and 3 market capitalization groupings (large-, mid-, and small-cap).

Examining the Fund’s most recent performance, Perényi is concerned that the Fund’s holdings, although approximately aligned with the STOXX Europe 600 Index’s country weights, may have unrecognized risk biases and concentrations. Perényi requests Elsa Lund, DIMCO’s chief risk officer, to investigate the Fund’s diversification. Lund asks her analysts for ideas on how Perényi’s request can be addressed and receives three suggestions:

- Suggestion 1 Estimate the country, industry, and market-cap exposures of each Fund holding, aggregate them, and compare the aggregate exposures to the benchmark’s exposures. Then, examine mismatches for evidence of unexpected biases or concentrations.
- Suggestion 2 Identify inherent groupings among fund holdings based on a broad set of eight numerical (operating and financial) measures related to the holdings’ characteristics. Then, examine the groupings for evidence of unexpected biases or concentrations.
- Suggestion 3 Regress the return of the Fund on a set of country equity market indexes and sector indexes based on the Fund’s benchmark. Then, examine the regression coefficients for evidence of unexpected biases or concentrations.

Lund has several questions for analyst Greg Kane about using one or more clustering machine learning algorithms in relation to addressing Perényi’s request.

Lund asks if any information needs to be specified for ML clustering algorithms no matter which one is used. Kane replies that only the distance measure that the algorithm will use and the hyperparameter, k , for k-means clustering need to be specified.

Lund further asks whether there would be an advantage to using k-means clustering as opposed to hierarchical clustering. Kane replies that in his opinion, hierarchical clustering is the more appropriate algorithm.

- 1 Which analyst suggestion is *most likely* to be implemented using machine learning?
 - A Suggestion 1.
 - B Suggestion 2.
 - C Suggestion 3.
- 2 Kane’s reply to Lund’s first question is:
 - A correct.
 - B not correct, because other hyperparameters must also be specified.

- C not correct, because the feature set for describing the holdings measure must also be specified.
- 3 In stating a preference for hierarchical clustering in his reply to Lund's second question, Kane *most likely* is giving consideration to:
- A the speed of the algorithms.
- B the dimensionality of the data set.
- C the need to specify the hyperparameter, k , in using a k-means algorithm.

Solution to 1:

B is correct. A machine learning clustering algorithm could be used to implement Suggestion 2. A and C are incorrect because Suggestions 1 and 3, respectively, can be addressed easily using traditional regression analysis.

Solution to 2:

C is correct. Beyond specifying a distance measure and the k for k-means, whichever clustering algorithm is selected, the feature set used to group holdings by similarities must also be specified. Operating and financial characteristics of the companies represented in the Fund's portfolio are examples of such features.

Solution to 3:

C is correct. The value of the hyperparameter, k , the number of distinct groups into which the STOXX Europe 600 Index can be segmented, is not known. Using a hierarchical algorithm, the sorting of observations into clusters will occur without any prior input on the analyst's part.

NEURAL NETWORKS, DEEP LEARNING NETS, AND REINFORCEMENT LEARNING

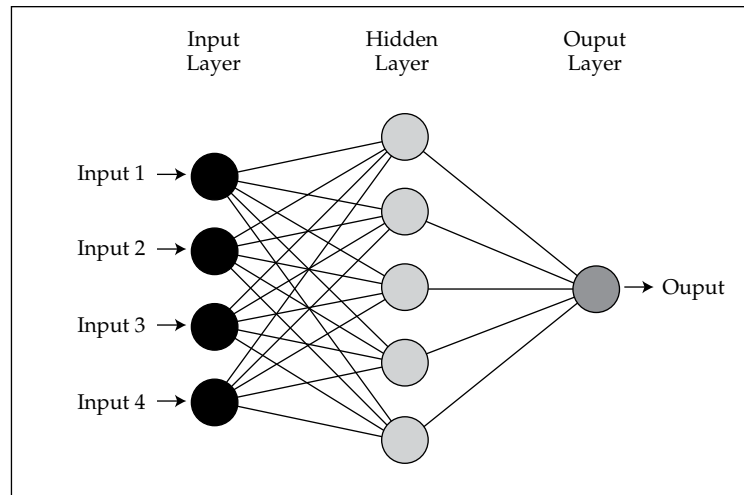
7

The artificial intelligence revolution has been driven in large part by advances in neural networks, deep learning algorithms, and reinforcement learning. These sophisticated algorithms can address highly complex machine learning tasks, such as image classification, face recognition, speech recognition, and natural language processing. These complicated tasks are characterized by non-linearities and interactions between large numbers of feature inputs. We now provide an overview of these algorithms and their investment applications.

7.1 Neural Networks

Neural networks (also called *artificial neural networks*, or ANNs) are a highly flexible type of ML algorithm that have been successfully applied to a variety of tasks characterized by non-linearities and complex interactions among features. Neural networks are commonly used for classification and regression supervised learning but are also important in reinforcement learning, which can be unsupervised.

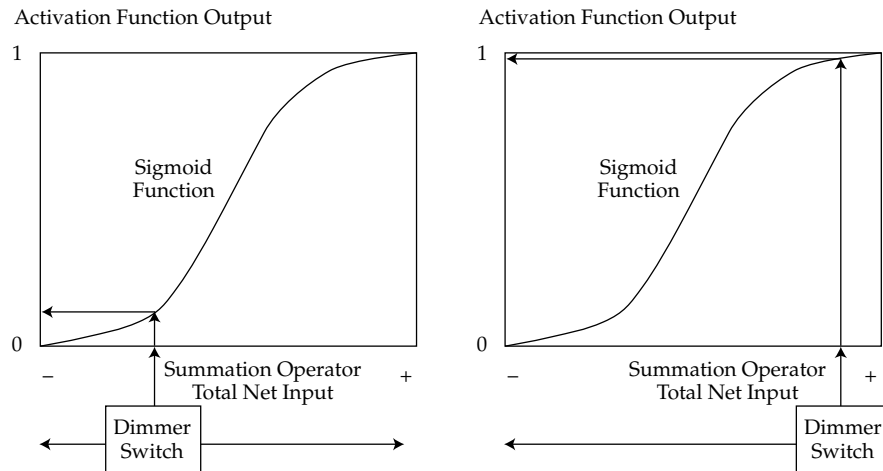
Exhibit 18 shows the connection between multiple regression and neural networks. Panel A represents a hypothetical regression for data using 4 inputs, the features x_1 to x_4 , and 1 output, the predicted value of the target variable y . Panel B shows a schematic representation of a basic neural network, which consists of *nodes* (circles) connected by *links* (arrows connecting nodes). Neural networks have three types of layers: an input layer (here with a node for each of the 4 features); hidden layers,

Exhibit 19 A More Complex (4-5-1) Neural Network with One Hidden Layer

Now consider any of the nodes to the right of the input layer. These nodes are sometimes called “neurons” because they process information received. Take the topmost hidden node. Four links connect to that node from the inputs, so the node gets four values transmitted by the links. Each node has, conceptually, two functional parts: a summation operator and an activation function. Once the node receives the four input values, the **summation operator** multiplies each value by a weight and sums the weighted values to form the total net input. The total net input is then passed to the **activation function**, which transforms this input into the final output of the node. Informally, the activation function operates like a light dimmer switch that decreases or increases the strength of the input. The activation function is characteristically non-linear, such as an S-shaped (sigmoidal) function (with output range of 0 to 1) or the rectified linear unit function shown in Panel B of Exhibit 18. Non-linearity implies that the rate of change of output differs at different levels of input.

This activation function is shown in Exhibit 20, where in the left graph a negative total net input is transformed via the S-shaped function into an output close to 0. This low output implies the node does not “fire,” so there is nothing to pass to the next node. Conversely, in the right graph a positive total net input is transformed into an output close to 1, so the node does fire. The output of the activation function is then transmitted to the next set of nodes if there is a second hidden layer or, as in this case, to the output layer node as the predicted value. The process of transmission just described (think of forward pointing arrows in Exhibit 19) is referred to as **forward propagation**.

Exhibit 20 Activation Function as “Light Dimmer Switch” at Each Node in a Neural Network



Starting with an initialized set of random network weights, training a neural network in a supervised learning context is an iterative process in which predictions are compared to actual values of labeled data and evaluated by a specified performance measure (e.g., mean squared error). Then, network weights are adjusted to reduce total error of the network. (If the process of adjustment works backward through the layers of the network, this process is called **backward propagation**). Learning takes place through this process of adjustment to network weights with the aim of reducing total error. Without proliferating notation relating to nodes, the gist of the updating can be expressed informally as:

$$\text{New weight} = (\text{Old weight}) - (\text{Learning rate}) \times (\text{Partial derivative of the total error with respect to the old weight}),$$

where “partial derivative” is a “gradient” or “rate of change of the total error with respect to the change in the old weight,” and **learning rate** is a parameter that affects the magnitude of adjustments. When learning is complete, all the network weights have assigned values.

The structure of a network in which all the features are interconnected with non-linear activation functions allows neural networks to uncover and approximate complex non-linear relationships among features. Broadly speaking, when more nodes and more hidden layers are specified, a neural network’s ability to handle complexity tends to increase (but so does the risk of overfitting).

Asset pricing is a noisy, stochastic process with potentially unstable relationships that challenge modelling processes, so researchers are asking if machine learning can improve our understanding of how markets work. Research comparing statistical and machine learning methods’ abilities to explain and predict equity prices so far indicates that simple neural networks produce models of equity returns at the individual stock and portfolio level that are superior to models built using traditional statistical methods due to their ability to capture dynamic and interacting variables. This suggests that ML-based models, such as neural networks, may simply be better able to cope with the non-linear relationships inherent in security prices. However, the trade-offs in using them are the lack of interpretability and the amount of data needed to train such models.

7.2 Deep Learning Nets

Neural networks with many hidden layers—at least 3 but often more than 20 hidden layers—are known as **deep learning nets** (DLNs) and are the backbone of the artificial intelligence revolution. Advances in DLNs have driven developments in many complex activities, such as image, pattern, and speech recognition. To state the operation of DLNs succinctly, they take a set of inputs x from a feature set (the input layer), which are then passed to a layer of non-linear mathematical functions (neurons) with weights w_{ij} (for neuron i and input j), each of which usually produces a scaled number in the range $(0, 1)$ or $(-1, 1)$. These numbers are then passed to another layer of functions and into another and so on until the final layer produces a set of probabilities of the observation being in any of the target categories (each represented by a node in the output layer). The DLN assigns the category based on the category with the highest probability. The DLN is trained on large data sets; during training, the weights w_i are determined to minimize a specified loss function.

In practice, while the number of nodes in the input and the output layers are typically determined by the characteristics of the features and predicted output, many model hyperparameters still must be decided, particularly the number of hidden nodes and their connectivity and activation architecture. The objective is to choose them to achieve the best out-of-sample performance, but it is still a challenge with no simple solution. As such, a good starting point is a ‘reasonable’ guess for hyperparameters based on experience and literature. The researcher can then observe the result and adjust the hyperparameters incrementally until the model performance goal is reached. In practice, DLNs require substantial time to train, and systematically varying the hyperparameters may not be feasible. So, for many problems with relatively small data sets, one can start with just two or three hidden layers and a few hundred nodes before tuning the parameters until a model with acceptable predictive power is achieved.

DLNs have been shown to be useful in general for pattern recognition problems (e.g., character and image recognition), credit card fraud detection, vision and control problems in autonomous cars, natural language processing (such as machine translation), and other applications. DLNs have become hugely successful because of a confluence of three developments: 1) advances in analytical methods for fitting these models; 2) the availability of large quantities of machine readable data to train models; and 3) fast computers, especially new chips in the graphics processing unit (GPU) class, tailored for the type of calculations done on DLNs.

Several financial firms are experimenting with DLNs for trading as well as automating their internal processes. Researchers have described how DLNs were trained to price options, mimicking the Black–Scholes–Merton model. This research used the same six input parameters for the model as input layer features—spot price, strike, time to maturity, dividend yield, risk-free interest rate, and volatility—with four hidden layers of 100 neurons each and one output layer. The predicted option prices out-of-sample were very close to the actual option prices: A regression of predicted option prices on actual prices had an R^2 of 99.8%.

Some researchers have used DLNs known as multi-layer perceptrons (or feed-forward networks) with many nodes (sometimes over 1,000) split over several hidden layers to predict corporate fundamental factors and price-related technical factors. Such DLNs typically use dozens of input features with many from company financial statements and also technical indicators of stock price momentum over various prior periods (e.g., 1, 3, 6 months). Some investment strategies using the DLN-predicted factors (e.g., book value to market value, operating income to market capitalization, etc.) have achieved compounded annual returns of 270 basis points more than returns using standard factor models.

7.3 Reinforcement Learning

Reinforcement learning (RL) is an algorithm that made headlines in 2017 when DeepMind's AlphaGo program beat the reigning world champion at the ancient game of Go. The RL algorithm involves an agent that should perform actions that will maximize its rewards over time, taking into consideration the constraints of its environment. In the case of AlphaGo, a virtual gamer (the agent) uses his/her console commands (the actions) with the information on the screen (the environment) to maximize his/her score (the reward). Unlike supervised learning, reinforcement learning has neither direct labeled data for each observation nor instantaneous feedback. With RL, the algorithm needs to observe its environment, learn by testing new actions (some of which may not be immediately optimal), and reuse its previous experiences. The learning subsequently occurs through millions of trials and errors. Academics and practitioners are applying RL in a similar way in investment strategies where the agent could be a virtual trader who follows certain trading rules (the actions) in a specific market (the environment) to maximize its profits (its reward). The success of RL in dealing with the complexities of financial markets is still an open question.

EXAMPLE 6

Deep Learning Nets

Glen Mitsui is the CIO for a large Australian state's Public Employees' Pension Fund (PEPF), which currently has assets under management (AUM) of A\$20 billion. The fund manages one-quarter of its assets internally, with A\$5 billion mostly in domestic government and corporate fixed-income instruments and domestic equities. The remaining three-quarters of AUM, or A\$15 billion, is managed by nearly 100 mostly active external asset managers and is invested in a wide range of asset classes, including foreign fixed income and equities, domestic and foreign hedge funds, REITs, commodities, and derivatives.

PEPF has a small staff of four investment professionals tasked with selecting and monitoring these external managers to whom it pays more than A\$400 million in fees annually. Performance (compared to appropriate benchmarks) of many of PEPF's external managers has been lagging over the past several years. After studying the situation, Mitsui concludes that style drift may be an important factor in explaining such underperformance, for which PEPF is not happy to pay. Mitsui believes that machine learning may help and consults with Frank Monroe, professor of data analysis at Epsilon University.

Monroe suggests using a deep learning net model that collects and analyzes the real-time trading data of PEPF's external managers and compares them to well-known investment styles (e.g., high dividend, minimum volatility, momentum, growth, value) to detect potential style drift. Mitsui arranges for Monroe to meet with PEPF's investment committee (IC) to discuss the matter. As a junior data analyst working with Monroe, you must help him answer the following requests from the IC:

- 1 Define a deep learning net.
- 2 Evaluate Monroe's opinion on the applicability of deep learning nets to Mitsui's problem.
- 3 Describe the functions of the three groups of layers of a deep learning net.

Solution to 1:

A deep learning net (DLN) is a neural network (NN) with many hidden layers (at least 3 but often more than 20). NNs and DLNs have been successfully applied to a wide variety of complex tasks characterized by non-linearities and interactions among features, particularly pattern recognition problems.

Solution to 2:

Mitsui wants to detect patterns of potential style drift in the daily trading of nearly 100 external asset managers in many markets. This task will involve the fast processing of huge amounts of complicated data. Monroe is correct that a DLN is well suited to PEPF's needs.

Solution to 3:

The input layer, the hidden layers, and the output layer constitute the three groups of layers of DLNs. The input layer receives the inputs (i.e., features) and has as many nodes as there are dimensions of the feature set. The hidden layers consist of nodes, each comprised of a summation operator and an activation function that are connected by links. These hidden layers are, in effect, where the model is learned. The final layer, the output layer, produces a set of probabilities of an observation being in any of the target style categories (each represented by a node in the output layer). The DLN assigns the category based on the style category with the highest probability.

EXAMPLE 7**Summing Up the Major Types of Machine Learning**

- 1 As used in supervised machine learning, regression problems involve:
 - A binary target variables.
 - B continuous target variables.
 - C categorical target variables.
- 2 Which of the following *best* describes penalized regression? Penalized regression:
 - A is unrelated to multiple linear regression.
 - B involves a penalty term that is added to the predicted target variable.
 - C is a category of general linear models used when the number of features and overfitting are concerns.
- 3 CART is *best* described as:
 - A an unsupervised ML algorithm.
 - B a clustering algorithm based on decision trees.
 - C a supervised ML algorithm that accounts for non-linear relationships among the features.
- 4 A neural network is *best* described as a technique for machine learning that is:
 - A exactly modeled on the human nervous system.
 - B based on layers of nodes connected by links when the relationships among the features are usually non-linear.

- C** based on a tree structure of nodes when the relationships among the features are linear.
- 5** Hierarchical clustering is *best* described as a technique in which:
 - A** the grouping of observations is unsupervised.
 - B** features are grouped into a pre-specified number, k , of clusters.
 - C** observations are classified according to predetermined labels.
- 6** Dimension reduction techniques are *best* described as a means to reduce a set of features:
 - A** to a manageable size without regard for the variation in the data.
 - B** to a manageable size while increasing the variation in the data.
 - C** to a manageable size while retaining as much of the variation in the data as possible.

Solution to 1:

B is correct. A and C are incorrect because when the target variable is binary or categorical, the problem is a *classification problem* rather than a regression problem.

Solution to 2:

C is correct. A is incorrect because penalized regression is related to multiple linear regression. B is incorrect because penalized regression involves adding a penalty term to the sum of the squared regression residuals.

Solution to 3:

C is correct. A is incorrect because CART is a supervised ML algorithm. B is incorrect because CART is a classification and regression algorithm, not a clustering algorithm.

Solution to 4:

B is correct. A is incorrect because neural networks are not exactly modeled on the human nervous system. C is incorrect because neural networks are not based on a tree structure of nodes when the relationships among the features are linear.

Solution to 5:

A is correct. B is incorrect because it refers to k-means clustering. C is incorrect because it refers to classification, which involves supervised learning.

Solution to 6:

C is correct because dimension reduction techniques, like PCA, are aimed at reducing the feature set to a manageable size while retaining as much of the variation in the data as possible.

SUMMARY

Machine learning methods are gaining usage at many stages in the investment management value chain. Among the major points made are the following:

- Machine learning aims at extracting knowledge from large amounts of data by learning from known examples to determine an underlying structure in the data. The emphasis is on generating structure or predictions without human intervention. An elementary way to think of ML algorithms is to “find the pattern, apply the pattern.”
- Supervised learning depends on having labeled training data as well as matched sets of observed inputs (X 's, or features) and the associated output (Y , or target). It can be divided into two categories: regression and classification. If the target variable to be predicted is continuous, then the task is one of regression. If the target variable is categorical or ordinal (e.g., determining a firm's rating), then it is a classification problem.
- With unsupervised learning, algorithms are trained with no labeled data, so they must infer relations between features, summarize them, or present an interesting underlying structure in their distributions that has not been explicitly provided. Two important types of problems well suited to unsupervised ML are dimension reduction and clustering.
- Another category of ML algorithm includes deep learning (based on neural networks) in which a computer learns from interacting with itself. Sophisticated algorithms address such highly complex tasks as image classification, face recognition, speech recognition and natural language processing, and reinforcement learning.
- Generalization describes the degree to which an ML model retains its explanatory power when predicting out-of-sample. Overfitting, a primary reason for lack of generalization, is the tendency of ML algorithms to tailor models to the training data at the expense of generalization to new data points.
- Bias error is the degree to which a model fits the training data. Variance error describes how much a model's results change in response to new data from validation and test samples. Base error is due to randomness in the data. Out-of-sample error equals bias error plus variance error plus base error.
- K-fold cross-validation is a technique for mitigating the holdout sample problem (excessive reduction of the training set size). The data (excluding test sample and fresh data) are shuffled randomly and then divided into k equal subsamples, with $k - 1$ samples used as training samples and one sample, the k th, used as a validation sample.
- Regularization describes methods that reduce statistical variability in high dimensional data estimation or prediction problems.
- LASSO (least absolute shrinkage and selection operator) is a popular type of penalized regression where the penalty term involves summing the absolute values of the regression coefficients. The greater the number of included features, the larger the penalty. So, a feature must make a sufficient contribution to model fit to offset the penalty from including it.
- Support vector machine (SVM) is a linear classifier that aims to seek the optimal hyperplane—the one that separates the two sets of data points by the maximum margin (and thus is typically used for classification).

- K-nearest neighbor (KNN) is a supervised learning technique most often used for classification. The idea is to classify a new observation by finding similarities (“nearness”) between it and its k-nearest neighbors in the existing data set.
- Classification and regression tree (CART) can be applied to predict either a categorical target variable, producing a classification tree, or a continuous target variable, producing a regression tree.
- A binary CART is a combination of an initial root node, decision nodes, and terminal nodes. The root node and each decision node represent a single feature (f) and a cutoff value (c) for that feature. The CART algorithm iteratively partitions the data into sub-groups until terminal nodes are formed that contain the predicted label.
- Ensemble learning is a technique of combining the predictions from a collection of models. It typically produces more accurate and more stable predictions than the best single model.
- A random forest classifier is a collection of many different decision trees generated by a bagging method or by randomly reducing the number of features available during training.
- Principal components analysis (PCA) is an unsupervised ML algorithm that reduces highly correlated features into fewer uncorrelated composite variables by transforming the feature covariance matrix. PCA produces eigenvectors that define the principal components (i.e., the new uncorrelated composite variables) and eigenvalues, which give the proportion of total variance in the initial data that is explained by each eigenvector and its associated principal component.
- K-means is an unsupervised ML algorithm that partitions observations into a fixed number (k) of non-overlapping clusters. Each cluster is characterized by its centroid, and each observation belongs to the cluster with the centroid to which that observation is closest.
- Hierarchical clustering is an unsupervised iterative algorithm that is used to build a hierarchy of clusters. Two main strategies are used to define the intermediary clusters (i.e., those clusters between the initial data set and the final set of clustered data).
- Agglomerative (bottom-up) hierarchical clustering begins with each observation being its own cluster. Then, the algorithm finds the two closest clusters, defined by some measure of distance, and combines them into a new, larger cluster. This process is repeated until all observations are clumped into a single cluster.
- Divisive (top-down) hierarchical clustering starts with all observations belonging to a single cluster. The observations are then divided into two clusters based on some measure of distance. The algorithm then progressively partitions the intermediate clusters into smaller clusters until each cluster contains only one observation.
- Neural networks consist of nodes connected by links. They have three types of layers: an input layer, hidden layers, and an output layer. Learning takes place in the hidden layer nodes, each of which consists of a summation operator and an activation function. Neural networks have been successfully applied to a variety of investment tasks characterized by non-linearities and complex interactions among variables.

- Neural networks with many hidden layers (at least 3 but often more than 20) are known as deep learning nets (DLNs) and are the backbone of the artificial intelligence revolution.
- The RL algorithm involves an agent that should perform actions that will maximize its rewards over time, taking into consideration the constraints of its environment.

REFERENCES

Bacham, Dinesh, and Janet Zhao. 2017. "Machine Learning: Challenges, Lessons, and Opportunities in Credit Risk Modeling." *Moody's Analytics Risk Perspectives* 9:28–35.

PRACTICE PROBLEMS

The following information relates to Questions 1–10

Alef Associates manages a long-only fund specializing in global smallcap equities. Since its founding a decade ago, Alef maintains a portfolio of 100 stocks (out of an eligible universe of about 10,000 stocks). Some of these holdings are the result of screening the universe for attractive stocks based on several ratios that use readily available market and accounting data; others are the result of investment ideas generated by Alef's professional staff of five securities analysts and two portfolio managers.

Although Alef's investment performance has been good, its Chief Investment Officer, Paul Moresanu, is contemplating a change in the investment process aimed at achieving even better returns. After attending multiple workshops and being approached by data vendors, Moresanu feels that data science should play a role in the way Alef selects its investments. He has also noticed that much of Alef's past outperformance is due to stocks that became takeover targets. After some research and reflection, Moresanu writes the following email to the Alef's CEO.

Subject: Investment Process Reorganization

I have been thinking about modernizing the way we select stock investments. Given that our past success has put Alef Associates in an excellent financial position, now seems to be a good time to invest in our future. What I propose is that we continue managing a portfolio of 100 global small-cap stocks but restructure our process to benefit from machine learning (ML). Importantly, the new process will still allow a role for human insight, for example, in providing domain knowledge. In addition, I think we should make a special effort to identify companies that are likely to be acquired. Specifically, I suggest following the four steps which would be repeated every quarter.

- Step 1 We apply ML techniques to a model including fundamental and technical variables (features) to predict next quarter's return for each of the 100 stocks currently in our portfolio. Then, the 20 stocks with the lowest estimated return are identified for replacement.
- Step 2 We utilize ML techniques to divide our investable universe of about 10,000 stocks into 20 different groups, based on a wide variety of the most relevant financial and non-financial characteristics. The idea is to prevent unintended portfolio concentration by selecting stocks from each of these distinct groups.
- Step 3 For each of the 20 different groups, we use labeled data to train a model that will predict the five stocks (in any given group) that are most likely to become acquisition targets in the next one year.

(Continued)

Step 4 Our five experienced securities analysts are each assigned four of the groups, and then each analyst selects their one best stock pick from each of their assigned groups. These 20 “high-conviction” stocks will be added to our portfolio (in replacement of the 20 relatively underperforming stocks to be sold in Step 1).

A couple of additional comments related to the above:

Comment 1 The ML algorithms will require large amounts of data. We would first need to explore using free or inexpensive historical datasets and then evaluate their usefulness for the ML-based stock selection processes before deciding on using data that requires subscription.

Comment 2 As time passes, we expect to find additional ways to apply ML techniques to refine Alef’s investment processes.

What do you think?
Paul Moresanu

-
- 1 The machine learning techniques appropriate for executing Step 1 are *most* likely to be based on:
 - A regression
 - B classification
 - C clustering
 - 2 Assuming regularization is utilized in the machine learning technique used for executing Step 1, which of the following ML models would be *least* appropriate:
 - A Regression tree with pruning.
 - B LASSO with lambda (λ) equal to 0.
 - C LASSO with lambda (λ) between 0.5 and 1.
 - 3 Which of the following machine learning techniques is *most* appropriate for executing Step 2:
 - A K-Means Clustering
 - B Principal Components Analysis (PCA)
 - C Classification and Regression Trees (CART)
 - 4 The hyperparameter in the ML model to be used for accomplishing Step 2 is?
 - A 100, the number of small-cap stocks in Alef’s portfolio.
 - B 10,000, the eligible universe of small-cap stocks in which Alef can potentially invest.
 - C 20, the number of different groups (i.e. clusters) into which the eligible universe of small-cap stocks will be divided.
 - 5 The target variable for the labelled training data to be used in Step 3 is *most* likely which one of the following?
 - A A continuous target variable.
 - B A categorical target variable.
 - C An ordinal target variable.

- 6 Comparing two ML models that could be used to accomplish Step 3, which statement(s) *best* describe(s) the advantages of using Classification and Regression Trees (CART) instead of K-Nearest Neighbor (KNN)?
- Statement I For CART there is no requirement to specify an initial hyperparameter (like K).
 - Statement II For CART there is no requirement to specify a similarity (or distance) measure.
 - Statement III For CART the output provides a visual explanation for the prediction.
- A Statement I only.
 - B Statement III only.
 - C Statements I, II and III.
- 7 Assuming a Classification and Regression Tree (CART) model is used to accomplish Step 3, which of the following is *most* likely to result in model overfitting?
- A Using the k-fold cross validation method
 - B Including an overfitting penalty (i.e., regularization term).
 - C Using a fitting curve to select a model with low bias error and high variance error.
- 8 Assuming a Classification and Regression Tree (CART) model is initially used to accomplish Step 3, as a further step which of the following techniques is most likely to result in more accurate predictions?
- A Discarding CART and using the predictions of a Support Vector Machine (SVM) model instead.
 - B Discarding CART and using the predictions of a K-Nearest Neighbor (KNN) model instead.
 - C Combining the predictions of the CART model with the predictions of other models – such as logistic regression, SVM, and KNN – via ensemble learning.
- 9 Regarding Comment #2, Moresanu has been thinking about the applications of neural networks (NNs) and deep learning (DL) to investment management. Which statement(s) *best* describe(s) the tasks for which NNs and DL are well-suited?
- Statement I NNs and DL are well-suited for image and speech recognition, and natural language processing.
 - Statement II NNs and DL are well-suited for developing single variable ordinary least squares regression models.
 - Statement III NNs and DL are well-suited for modelling non-linearities and complex interactions among many features.
- A Statement II only.
 - B Statements I and III.
 - C Statements I, II and III.
- 10 Regarding neural networks (NNs) that Alef might potentially implement, which of the following statements is *least* accurate?
- A NNs must have at least 10 hidden layers to be considered deep learning nets.

- B** The activation function in a node operates like a light dimmer switch since it decreases or increases the strength of the total net input.
- C** The summation operator receives input values, multiplies each by a weight, sums up the weighted values into the total net input, and passes it to the activation function.

SOLUTIONS

- 1 A is correct. The target variable (quarterly return) is continuous, hence this calls for a supervised machine learning based regression model.
B is incorrect, since classification uses categorical or ordinal target variables, while in Step 1 the target variable (quarterly return) is continuous.
C is incorrect, since clustering involves unsupervised machine learning so does not have a target variable.
- 2 B is correct. It is least appropriate because with LASSO, when $\lambda = 0$ the penalty (i.e., regularization) term reduces to zero, so there is no regularization and the regression is equivalent to an ordinary least squares (OLS) regression.
A is incorrect. With Classification and Regression Trees (CART), one way that regularization can be implemented is via pruning which will reduce the size of the regression tree—sections that provide little explanatory power are pruned (i.e., removed).
C is incorrect. With LASSO, when λ is between 0.5 and 1 the relatively large penalty (i.e., regularization) term requires that a feature makes a sufficient contribution to model fit to offset the penalty from including it in the model.
- 3 A is correct. K-Means clustering is an unsupervised machine learning algorithm which repeatedly partitions observations into a fixed number, k , of non-overlapping clusters (i.e., groups).
B is incorrect. Principal Components Analysis is a long-established statistical method for dimension reduction, not clustering. PCA aims to summarize or reduce highly correlated features of data into a few main, uncorrelated composite variables.
C is incorrect. CART is a supervised machine learning technique that is most commonly applied to binary classification or regression.
- 4 C is correct. Here, 20 is a hyperparameter (in the K-Means algorithm), which is a parameter whose value must be set by the researcher before learning begins.
A is incorrect, because it is not a hyperparameter. It is just the size (number of stocks) of Alef's portfolio.
B is incorrect, because it is not a hyperparameter. It is just the size (number of stocks) of Alef's eligible universe.
- 5 B is correct. To predict which stocks are likely to become acquisition targets, the ML model would need to be trained on categorical labelled data having the following two categories: "0" for "not acquisition target", and "1" for "acquisition target".
A is incorrect, because the target variable is categorical, not continuous.
C is incorrect, because the target variable is categorical, not ordinal (i.e., 1st, 2nd, 3rd, etc.).
- 6 C is correct. The advantages of using CART over KNN to classify companies into two categories ("not acquisition target" and "acquisition target"), include all of the following: For CART there are no requirements to specify an initial hyperparameter (like K) or a similarity (or distance) measure as with KNN, and CART provides a visual explanation for the prediction (i.e., the feature variables and their cut-off values at each node).
A is incorrect, because CART provides all of the advantages indicated in Statements I, II and III.

B is incorrect, because CART provides all of the advantages indicated in Statements I, II and III.

- 7 C is correct. A fitting curve shows the trade-off between bias error and variance error for various potential models. A model with low bias error and high variance error is, by definition, overfitted.

A is incorrect, because there are two common methods to reduce overfitting, one of which is proper data sampling and cross-validation. K-fold cross validation is such a method for estimating out-of-sample error directly by determining the error in validation samples.

B is incorrect, because there are two common methods to reduce overfitting, one of which is preventing the algorithm from getting too complex during selection and training, which requires estimating an overfitting penalty.

- 8 C is correct. Ensemble learning is the technique of combining the predictions from a collection of models, and it typically produces more accurate and more stable predictions than the best single model.

A is incorrect, because a single model will have a certain error rate and will make noisy predictions. By taking the average result of many predictions from many models (i.e., ensemble learning) one can expect to achieve a reduction in noise as the average result converges towards a more accurate prediction.

B is incorrect, because a single model will have a certain error rate and will make noisy predictions. By taking the average result of many predictions from many models (i.e., ensemble learning) one can expect to achieve a reduction in noise as the average result converges towards a more accurate prediction.

- 9 B is correct. NNs and DL are well-suited for addressing highly complex machine learning tasks, such as image classification, face recognition, speech recognition and natural language processing. These complicated tasks are characterized by non-linearities and complex interactions between large numbers of feature inputs.

A is incorrect, because NNs and DL are well-suited for addressing highly complex machine learning tasks, not simple single variable OLS regression models.

C is incorrect, because NNs and DL are well-suited for addressing highly complex machine learning tasks, not simple single variable OLS regression models.

- 10 A is correct. It is the least accurate answer because neural networks with many hidden layers—at least 3, but often more than 20 hidden layers—are known as deep learning nets.

B is incorrect, because the node's activation function operates like a light dimmer switch which decreases or increases the strength of the (total net) input.

C is incorrect, because the node's summation operator multiplies each (input) value by a weight and sums up the weighted values to form the total net input. The total net input is then passed to the activation function.